VOL. 15

API design

Designing, documenting, testing and consuming HTTP APIs you can be proud of

Source	MDN HTTP + OpenAPI Specification (fallback for rapidapi.com/learn)
Generated	2026-10-10
Level	Beginner
Estimated study time	25 hours
Language	English
Volume	Vol. 15 - API

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 8

Chapter map 8

Chapter 1 - What an API is 9

1.1 Summary 9

1.2 Concepts 10

The client, the server and the contract 10

Anatomy of a request 10

Anatomy of a response 11

Why HTTP and not something custom? 11

1.3 Examples 11

1.4 Common mistakes and best practices 14

1.5 Practice exercises 14

Solutions 14

1.6 Mini test 14

Answer key and explanations 15

Chapter 2 - REST principles and constraints 15

2.1 Summary 15

2.2 Concepts 16

Resources and representations 16

The six constraints 16

The Richardson maturity model 17

2.3 Examples 17

2.4 Common mistakes and best practices 19

2.5 Practice exercises 20

Solutions 20

2.6 Mini test 20

Answer key and explanations 21

Chapter 3 - Designing resources and URIs 21

3.1 Summary 21

3.2 Concepts 21

Nouns, not verbs 21

Plurals, singulars and identifiers 22

When to nest, and when to stop . . . 22
Naming rules that never go out of style . . . 22
3.3 Examples . . . 23
3.4 Common mistakes and best practices . . . 26
3.5 Practice exercises . . . 26
Solutions . . . 26
3.6 Mini test . . . 27
Answer key and explanations . . . 27
Chapter 4 - HTTP methods done right . . . 27
4.1 Summary . . . 27
4.2 Concepts . . . 28
Safe and idempotent . . . 28
`PUT` against `PATCH` . . . 28
`POST`, retries and the idempotency key . . . 29
`HEAD` and `OPTIONS` . . . 29
4.3 Examples . . . 29
4.4 Common mistakes and best practices . . . 32
4.5 Practice exercises . . . 33
Solutions . . . 33
4.6 Mini test . . . 33
Answer key and explanations . . . 33
Chapter 5 - Status codes and error payloads . . . 34
5.1 Summary . . . 34
5.2 Concepts . . . 34
The families . . . 34
The shortlist worth memorising . . . 34
One error shape for everything . . . 35
5.3 Examples . . . 36
5.4 Common mistakes and best practices . . . 39
5.5 Practice exercises . . . 39
Solutions . . . 40
5.6 Mini test . . . 40
Answer key and explanations . . . 40
Chapter 6 - Requests, responses and JSON hygiene . . . 41
6.1 Summary . . . 41

6.2 Concepts . . . 41
The two sides of the exchange . . . 41
Naming and typing rules . . . 41
`null` versus an absent key . . . 42
The envelope . . . 42
6.3 Examples . . . 42
6.4 Common mistakes and best practices . . . 44
6.5 Practice exercises . . . 45
Solutions . . . 45
6.6 Mini test . . . 46
Answer key and explanations . . . 46
Chapter 7 - Filtering, sorting and query design . . . 46
7.1 Summary . . . 46
7.2 Concepts . . . 47
The vocabulary of a collection endpoint . . . 47
Filtering shapes . . . 47
Sorting and stability . . . 47
Validation, not silent tolerance . . . 48
7.3 Examples . . . 48
7.4 Common mistakes and best practices . . . 52
7.5 Practice exercises . . . 53
Solutions . . . 53
7.6 Mini test . . . 53
Answer key and explanations . . . 54
Chapter 8 - Pagination . . . 54
8.1 Summary . . . 54
8.2 Concepts . . . 54
Offset pagination . . . 54
Cursor pagination . . . 55
The `Link` header . . . 55
8.3 Examples . . . 55
8.4 Common mistakes and best practices . . . 60
8.5 Practice exercises . . . 61
Solutions . . . 61
8.6 Mini test . . . 61

Answer key and explanations . . . 61
Chapter 9 - Authentication and authorization . . . 62
9.1 Summary . . . 62
9.2 Concepts . . . 62
Keys, tokens and sessions . . . 62
How a JWT is built . . . 62
The claims worth knowing . . . 63
OAuth 2.0 in one picture . . . 63
9.3 Examples . . . 64
9.4 Common mistakes and best practices . . . 67
9.5 Practice exercises . . . 68
Solutions . . . 68
9.6 Mini test . . . 68
Answer key and explanations . . . 68
Chapter 10 - Documenting with OpenAPI . . . 69
10.1 Summary . . . 69
10.2 Concepts . . . 69
The skeleton . . . 69
What an operation declares . . . 69
`$ref`, the reason OpenAPI stays small . . . 70
10.3 Examples . . . 70
10.4 Common mistakes and best practices . . . 74
10.5 Practice exercises . . . 75
Solutions . . . 75
10.6 Mini test . . . 75
Answer key and explanations . . . 75
Chapter 11 - Versioning, rate limiting and cross-origin security . . . 76
11.1 Summary . . . 76
11.2 Concepts . . . 76
What counts as a breaking change . . . 76
Where the version lives . . . 77
Rate limiting . . . 77
CORS in one exchange . . . 78
11.3 Examples . . . 78
11.4 Common mistakes and best practices . . . 82

11.5 Practice exercises . . . 83
Solutions . . . 83
11.6 Mini test . . . 83
Answer key and explanations . . . 84
Chapter 12 - Testing APIs . . . 84
12.1 Summary . . . 84
12.2 Concepts . . . 84
The test pyramid for an API . . . 85
A good assertion . . . 85
Isolating the server . . . 85
12.3 Examples . . . 85
12.4 Common mistakes and best practices . . . 89
12.5 Practice exercises . . . 90
Solutions . . . 90
12.6 Mini test . . . 90
Answer key and explanations . . . 91
Chapter 13 - Consuming third-party APIs . . . 91
13.1 Summary . . . 91
13.2 Concepts . . . 91
The wrapper pattern . . . 91
Timeouts, retries and backoff . . . 92
13.3 Examples . . . 92
13.4 Common mistakes and best practices . . . 95
13.5 Practice exercises . . . 95
Solutions . . . 96
13.6 Mini test . . . 96
Answer key and explanations . . . 96
Chapter 14 - Capstone: design, document and test a bookmarks API . . . 97
14.1 Summary . . . 97
14.2 Concepts . . . 97
The brief . . . 97
The resource model . . . 97
The decisions, and where they came from . . . 97
14.3 Examples . . . 98

14.4 Common mistakes and best practices . . . 100
14.5 Practice exercises . . . 101
Solutions . . . 101
14.6 Mini test . . . 102
Answer key and explanations . . . 102
Cheat sheet . . . 102
15.1 Requests and responses . . . 102
15.2 Methods . . . 102
15.3 Status codes . . . 103
15.4 Problem documents . . . 103
15.5 Query parameters . . . 103
15.6 Pagination envelopes . . . 104
15.7 Security and quota headers . . . 104
15.8 CORS and security headers . . . 104
15.9 OpenAPI skeleton . . . 105
15.10 Client recipe . . . 105
Final test . . . 105
16.1 Final test answers . . . 106
Sources . . . 107

How to use this volume

Prerequisites. Working modern JavaScript (Vol. 03): `let` and `const`, arrow functions, template literals, destructuring, `async`/`await`, modules, and a first look at `fetch`. A little JSON practice helps too. No server experience is assumed: everything runs on your own machine with Node.js only.

Estimated study time. About 25 hours: two hours per chapter plus the time you spend on your own experiments. Chapters 1 to 8 are the design core, chapters 9 to 13 cover the parts that make an API public and reliable, and chapter 14 puts all of it together in one small but complete service.

How the chapters work. Every chapter opens with a summary, explains the concepts with tables and ASCII diagrams, shows numbered examples with the output they really produced, lists the common mistakes with the best practice for each, and ends with practice exercises, their solutions, and a mini test with an answer key.

How to run the code. Every HTTP transcript in this volume was produced by a dependency-free Node server that lives in three files: `app.mjs` (the routes), `server.mjs` (the listener) and `bodies/*.json` (request payloads). Start it, then talk to it with `curl.exe`:

shell

```shell
mkdir api-check
cd api-check
node server.mjs
curl.exe -s -i http://127.0.0.1:3000/api/v1/health
```

output

```text
bookmarks API listening on http://127.0.0.1:3000
```

The server keeps its data in memory, so a restart resets the collection. Blocks tagged `js` run with Node, blocks tagged `bash` are terminal commands, blocks tagged `text` are real terminal output, blocks tagged `http` show a request and response pair, and blocks tagged `json` and `yaml` are data.

About the source. The syllabus in `plan/15-api.md` was meant to follow a course hub that now sits behind a login wall and answers with a 404, so this volume is built from the free, official documentation that replaces it: the MDN Web Docs HTTP material, RFC 9110 for HTTP semantics, RFC 9457 for problem details, and the OpenAPI Specification for descriptions. The example API is original work written for this volume, and no text was copied from those sites.

Chapter map

#	Chapter	You will learn
1	What an API is	the client/server split, the anatomy of a request and a response
2	REST principles and constraints	resources, representations and the six constraints of REST

#	Chapter	You will learn
3	Designing resources and URIs	nouns, plural collections, nesting depth, path parameters
4	HTTP methods done right	safe and idempotent methods, the method matrix, PUT against PATCH
5	Status codes and error payloads	the status code families and one consistent error shape
6	Requests, responses and JSON hygiene	content types, headers, camelCase, enums, null versus absent
7	Filtering, sorting and query design	query parameters, search terms, stable ordering, validation
8	Pagination	offset against cursor pages, `Link` headers, page envelopes
9	Authentication and authorization	API keys, JWT structure, OAuth 2.0 and the authorization code flow
10	Documenting with OpenAPI	a real OpenAPI 3.1 document, schemas, references, linting
11	Versioning, rate limiting and cross-origin security	URI against header versioning, quotas, CORS preflight, headers
12	Testing APIs	manual checks with curl and an automated suite with `node:test`
13	Consuming third-party APIs	timeouts, retries, exponential backoff, `Retry-After`
14	Capstone: the bookmarks API	design, document, test and run one coherent public API
-	Cheat sheet	compact reference for the whole volume
-	Final test	25 questions with answers and explanations

Chapter 1 - What an API is

1.1 Summary

- An API is a contract that lets two programs talk without knowing how the other one is built.
- A client sends a request; a server reads it, does something and sends back a response.
- HTTP is the transport most web APIs use: it is text based, stateless and already understood by every browser.
- A request has a method, a target path, headers and usually a body.
- A response has a status code, headers and usually a body.

- Headers carry metadata about the message, the client, the response and the policy in force.
- JSON became the default body format because both JavaScript and every server language can read and write it.
- The demo bookmarks API used throughout this volume is started with one command and answers on `http://127.0.0.1:3000`.
- Good APIs are predictable: the same input always produces the same shape of output.
- An API is a user interface, only its users are programs instead of people.

1.2 Concepts

The client, the server and the contract

Two programs need to exchange data. The one that starts the exchange is the client; the one that answers is the server. The API is the set of rules they agree on: which addresses exist, which verbs are allowed on them, which headers are understood, and what the bodies look like.

output

```
client                              server
------                              ------
build a request
  method  GET /api/v1/bookmarks
  headers Accept, X-API-Key
  body    (none)
                |
                |   sends bytes over TCP
                v
                                    read the request line
                                    read the headers
                                    match a route
                                    run the handler
                                    build a response
                |
                |   sends bytes back
                v
read the response
  status  200 OK
  headers Content-Type, X-RateLimit-*
  body    {"data":[...],"meta":{...}}
```

The client does not know whether the server keeps its bookmarks in PostgreSQL, in memory or on stone tablets. The server does not know whether the client is a browser, a phone app or a cron job. That ignorance is the point: both sides can change as long as the contract holds.

Anatomy of a request

Part	Example	Meaning
Method	`GET`	what the client wants done
Target	`/api/v1/bookmarks?limit=2`	which resource, plus query parameters

Part	Example	Meaning
Version	`HTTP/1.1`	the protocol version in the request line
Header	`Accept: application/json`	metadata about the client and the message
Body	`{"title":"RFC 9110"}`	the payload, usually JSON, absent on `GET`

A request line looks like this once it is written out:

HTTP

```
GET /api/v1/bookmarks?limit=2 HTTP/1.1
Host: 127.0.0.1:3000
Accept: application/json
X-API-Key: lf-demo-key-123
```

Anatomy of a response

Part	Example	Meaning
Status line	`HTTP/1.1 200 OK`	a three digit code plus a reason phrase
Status class	`2xx` success, `4xx` client error, `5xx` server error	the family of the answer
Header	`Content-Type: application/json; charset=utf-8`	how to read the body
Header	`X-RateLimit-Remaining: 98`	policy information
Body	`{"data":[...],"meta":{...}}`	the representation of the resource

Why HTTP and not something custom?

HTTP already solves the boring parts of remote communication: a standard way to address resources, verbs with agreed meanings, a status vocabulary, a header mechanism that proxies and caches understand, and content negotiation. Writing your own protocol over a socket means re-implementing all of that, and then debugging it.

What HTTP does not give you is agreement on your resource names, your field names, your error shape or your pagination style. That is what API design is about, and it is the subject of this volume.

1.3 Examples

Example 1: start the demo server.

shell

```
node server.mjs
```

output

```
bookmarks API listening on http://127.0.0.1:3000
```

The server prints one line and then waits. Nothing else is installed: the routes use `node:http` from the standard library.

Example 2: a complete exchange, request and response.

shell

```
curl.exe -s -i http://127.0.0.1:3000/api/v1/health
```

HTTP

```
HTTP/1.1 200 OK
X-Request-Id: 799e7c85-7151-4a96-91e6-aa131c669431
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 99
X-RateLimit-Reset: 1791669942
Content-Type: application/json; charset=utf-8
Content-Length: 67
Date: Sat, 10 Oct 2026 22:04:41 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "status": "ok",
  "version": "1.0.0",
  "uptimeSeconds": 577
}
```

`-s` silences the progress meter and `-i` includes the response headers, so the block above is the whole exchange. Read it from the top: a status line, eleven headers, a blank line, then the body.

Example 3: the parts of the request line.

JavaScript

```
const url = new URL("https://api.example.com/api/v1/bookmarks?limit=2&tag=http");
console.log("pathname   :", url.pathname);
console.log("limit      :", url.searchParams.get("limit"));
console.log("tag        :", url.searchParams.get("tag"));
console.log("all keys   :", [...url.searchParams.keys()].join(", "));
url.searchParams.set("offset", "2");
console.log("with offset:", url.toString());
```

output

```
pathname   : /api/v1/bookmarks
limit      : 2
tag        : http
all keys   : limit, tag
with offset: https://api.example.com/api/v1/bookmarks?limit=2&tag=http&offset=2
```

The path identifies the resource. The query string adjusts how the resource is returned; it never creates a new resource.

Example 4: a request with a body needs a content type.

shell

```
curl.exe -s -i -X POST http://127.0.0.1:3000/api/v1/bookmarks \
  -H "Content-Type: application/json" -H "X-API-Key: lf-demo-key-123" \
  --data "@bodies/missing-title.json"
```

HTTP

```
HTTP/1.1 400 Bad Request
X-Request-Id: 7b0a2e48-14e0-458f-8181-5d26465b904b
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 98
X-RateLimit-Reset: 1791669960
Content-Type: application/problem+json; charset=utf-8
Content-Length: 325
Date: Sat, 10 Oct 2026 22:04:59 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "type": "https://learnfree.dev/problems/validation-error",
  "title": "Request body failed validation",
  "status": 400,
  "detail": "1 field is invalid",
  "instance": "/api/v1/bookmarks",
  "errors": [
    {
      "field": "title",
      "code": "required",
      "message": "a non empty title is required"
    }
  ]
}
```

The request body is read from a file so that the JSON stays readable on the command line; `bodies/missing-title.json` holds `{"url":"https://example.com/no-title"}`. The server answers `400` because the payload is valid JSON but invalid data.

Example 5: the same contract seen from a browser.

JavaScript

```
const response = await fetch("http://127.0.0.1:3000/api/v1/bookmarks?limit=2", {
  headers: { Accept: "application/json" },
});
console.log(response.status);
console.log(response.headers.get("content-type"));
const page = await response.json();
console.log(page.data.length, "bookmarks, total", page.meta.total);
```

output

```
200
application/json; charset=utf-8
2 bookmarks, total 12
```

`fetch` returns a response object first and reads the body on demand, which is why `response.json()` is awaited separately.

1.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Treating the API as an implementation detail	every client breaks on each refactor	publish the contract and version it
Inventing a binary protocol over sockets	no tooling, no caching, no proxies	use HTTP and JSON unless you have a hard reason not to
Putting action words in the path (`/getBookmarks`)	the URL stops describing data	name resources, let the verb be the method
Trusting the request body without checking it	crashes, corrupt rows, security holes	validate every field and return `400` with details
Returning different shapes for the same route	clients write branches for every case	one response schema per route, always
Forgetting `Content-Type` on a POST	the server cannot read the body	send `application/json` with every JSON body
Ignoring response headers	you miss quota and rate information	read `X-RateLimit-*` and `Retry-After`

- Sketch the exchange on paper before writing a route: method, path, status, body.
- When a response surprises you, read the headers first; the body rarely contradicts them.

1.5 Practice exercises

1. Name the four parts of an HTTP request and the three parts of a response.
2. What is the difference between the path and the query string of a URL?
3. Which header tells the client how to interpret the body of a response?
4. Why does a `GET` request normally have no body?
5. In example 2, what does `X-RateLimit-Remaining: 99` tell a client?

Solutions

1. Request: method, target, headers, body. Response: status line, headers, body.
2. The path names the resource (`/api/v1/bookmarks`); the query string modifies how it is returned (`?limit=2`). Adding a query parameter never creates a new resource.
3. `Content-Type`, for example `application/json; charset=utf-8`.
4. Because `GET` means "read this resource": it is safe, so it must not change anything, and a body would suggest a payload to process.
5. That 99 requests are left in the current quota window, so the client should slow down before the budget runs out instead of waiting for a `429`.

1.6 Mini test

1. Which side of an HTTP exchange sends the request? A) the server B) the client C) the proxy D) the load balancer
2. Which status class means "your request was wrong"? A) `1xx` B) `2xx` C) `4xx` D) `5xx`
3. True or false: a URL query string identifies a different resource from the same path without it.
4. Which part of a response tells the client how to parse the body? A) `Content-Length` B) `Content-Type` C) `Date` D) `Connection`
5. What does the demo server print when it starts successfully?
6. Which header in example 2 is non-standard but useful for quota tracking? A) `Date` B) `Connection` C) `X-RateLimit-Remaining` D) `Vary`
7. Why is `{"status":"ok"}` a sensible body for a health endpoint? A) it is small and stable, so monitoring can compare it B) it compresses well C) it is the only format HTTP allows D) it avoids headers
8. Which method would you use to ask for the same health document again without changing anything?

Answer key and explanations

1. B - the client always starts the exchange; the server answers.
2. C - `4xx` codes report a problem with the request, `5xx` a problem on the server.
3. False - the same resource is served with different options applied; query parameters change the representation, not the identity.
4. B - `Content-Type` names the media type and charset of the body.
5. `bookmarks API listening on http://127.0.0.1:3000`.
6. C - `X-` headers are convention based extensions; they are widely used for quota, tracing and debugging information.
7. A - monitoring only needs a stable, tiny document it can compare against an expectation.
8. `GET`, because it is safe: repeating it never changes the resource.

Chapter 2 - REST principles and constraints

2.1 Summary

- REST is an architectural style for distributed systems, described by Roy Fielding in his doctoral dissertation.
- A REST system is built around resources, each addressed by its own URI.
- Clients exchange representations of resources, most often JSON documents.
- The six REST constraints are client-server, statelessness, cacheability, a uniform interface, a layered system and optional code on demand.
- Statelessness means every request carries everything the server needs to answer it.

- HTTP satisfies most of REST already: it has verbs, status codes, headers and caching rules.
- The uniform interface says the same method and the same path always mean the same thing.
- The Richardson maturity model grades an API from plain HTTP tunnels up to hypermedia controls.
- An API can be "REST-ish" and still be excellent; the constraints are guidance, not a religion.
- Predictability matters more than purity: a boring API that never surprises anyone beats a clever one.

2.2 Concepts

Resources and representations

A resource is anything worth naming: a bookmark, a collection of bookmarks, a search result, a user profile. The resource itself lives on the server; the client only ever sees a representation of it, for example a JSON object or a CSV export of the same rows.

output

```
   resource (server side)          representation (on the wire)
+----------------------+        +-------------------------------+
| bm_003               | -----> | application/json              |
|  url, title, tags    |        | {"id":"bm_003","title":...}   |
|  created 2026-09-03  | -----> | application/xml (if you want) |
+----------------------+        +-------------------------------+
```

Because the client holds a representation and not the resource, two clients can ask for two different formats of the same thing without the server duplicating data.

The six constraints

Constraint	What it demands	What it buys you
Client-server	separation of concerns	the UI and the storage can evolve apart
Statelessness	no session state on the server between requests	simple scaling, easy retries
Cacheability	responses declare whether they may be reused	fewer requests, less load
Uniform interface	the same rules for every resource	clients learn once, apply everywhere
Layered system	the client cannot tell a proxy from the origin	load balancers, gateways, CDNs
Code on demand	servers may ship executable code	rarely used in JSON APIs

Statelessness in practice. The server never remembers that you just asked for page one. If page two needs a page size, the client sends it again. That is why query parameters and cursors appear in every request instead of being kept in a server-side session.

Cacheability in practice. A response says either "reuse me" or "do not reuse me". The demo API is deliberately the second kind for tokens:

HTTP

```
Cache-Control: no-store
```

Code on demand is the one constraint most web APIs skip, and skipping it is normal.

The Richardson maturity model

A quick way to grade how "REST" an API is:

Level	Name	What it looks like
0	The swamp of POX	one endpoint, one verb, an action name in the body
1	Resources	many endpoints, still one verb (`POST /bookmarks/create`)
2	HTTP verbs	`GET`/`POST`/`PATCH`/`DELETE` plus meaningful status codes
3	Hypermedia controls	responses link to related resources and next pages

Most production APIs sit at level 2, and that is where the practical value is. Level 3 is elegant but needs a media type that understands links.

2.3 Examples

Example 1: one resource, two representations.

JSON

```
{
  "id": "bm_003",
  "url": "https://www.rfc-editor.org/rfc/rfc9110.html",
  "title": "RFC 9110",
  "description": "HTTP Semantics, the normative method rules",
  "tags": ["http", "rfc"],
  "favorite": false,
  "createdAt": "2026-09-03T08:40:00.000Z"
}
```

JSON

```
{
  "type": "bookmark",
  "id": "bm_003",
  "attributes": {
    "url": "https://www.rfc-editor.org/rfc/rfc9110.html",
    "title": "RFC 9110"
  }
}
```

Both documents describe the same bookmark. The first is flat and simple; the second separates type, id and attributes, the style `jsonapi.org` documents. Choose one and keep it.

Example 2: statelessness, shown on the wire.

shell

```
curl.exe -s -i http://127.0.0.1:3000/api/v1/bookmarks?limit=1
```

HTTP

```
HTTP/1.1 200 OK
X-Request-Id: 65d6d8a2-848a-4bc7-9dc8-be0ce2840d68
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 93
X-RateLimit-Reset: 1791669942
Content-Type: application/json; charset=utf-8
Content-Length: 762
Link: <http://127.0.0.1:3000/api/v1/bookmarks?limit=2&cursor=eyJvZmZzZXQiOjR9>; rel="next"
Date: Sat, 10 Oct 2026 22:04:51 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "data": [
    {
      "id": "bm_003",
      "url": "https://www.rfc-editor.org/rfc/rfc9110.html",
      "title": "RFC 9110",
      "description": "HTTP Semantics, the normative method rules",
      "tags": [
        "http",
        "rfc"
      ],
      "favorite": false,
      "createdAt": "2026-09-03T08:40:00.000Z"
    }
  ],
  "meta": {
    "limit": 1,
    "offset": 0,
    "total": 12,
    "hasMore": true,
    "nextCursor": "eyJvZmZzZXQiOjF9"
  }
}
```

There is no `Set-Cookie` header and no session id, and the `Link` header carries the full URL of the next page instead of relying on the server to remember where you are.

Example 3: the uniform interface, one path with different verbs.

shell

```
curl.exe -s -o NUL -w '%{http_code}\n' http://127.0.0.1:3000/api/v1/bookmarks
curl.exe -s -o NUL -w '%{http_code}\n' -X DELETE http://127.0.0.1:3000/api/v1/bookmarks
curl.exe -s -o NUL -w '%{http_code}\n' -X POST http://127.0.0.1:3000/api/v1/bookmarks
```

output

```
200
405
401
```

`GET /api/v1/bookmarks` reads the collection (`200`). `DELETE` on the same path has no meaning, because there is no "the collection" object to delete, so the server answers `405` and lists the methods it does accept. `POST` to the same path means "create one", and the server asks for credentials before doing anything (`401`).

Example 4: what a level 1 API would look like instead.

HTTP

```http
POST /api HTTP/1.1
Content-Type: application/json

{"action":"listBookmarks","limit":1}
```

HTTP

```http
HTTP/1.1 200 OK
Content-Type: application/json

{"result":[{"id":"bm_003","title":"RFC 9110"}]}
```

One endpoint, one verb, an action name in the body. It works, but nothing about it is discoverable: caches cannot help, proxies cannot inspect it, and every new action needs a new string in a switch statement.

Example 5: the same idea expressed at level 2.

shell

```shell
curl.exe -s "http://127.0.0.1:3000/api/v1/bookmarks?limit=1" -o page.json
node -e "const p=require('./page.json');console.log(p.data[0].title)"
```

output

```
MDN HTTP reference
```

The verb is the intent (`GET`), the path is the noun (`/api/v1/bookmarks`), and the options live in the query string (`?limit=1`).

2.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Storing session state on the server	every request needs one specific node	keep requests self contained
Verbs in the path (`/getBookmarks`)	the URL stops describing data	name resources, use the method
Returning 200 with an error inside the body	proxies and clients think it worked	use the status code first, the body second
One giant endpoint that dispatches on an action field	no caching, no tooling, no contract	one resource per path
Caching responses that change per user	users see each other's data	`Cache-Control: private, no-store` for personal data

Mistake	Why it hurts	Best practice
Treating REST as a checklist of purity	you spend the budget on the wrong thing	be consistent, be boring, be documented

- Decide early which headers are your contract; clients will hard-code them.
- When in doubt, ask what a cache, a proxy or a log reader would need to know.

2.5 Practice exercises

1. Explain statelessness in one sentence, and name one thing it makes harder.
2. What is the difference between a resource and a representation?
3. Give one benefit of cacheability and one situation where it must be disabled.
4. Grade this endpoint with the Richardson model: `POST /api {"action":"deleteBookmark","id":"bm_003"}`.
5. Why does `Cache-Control: no-store` make sense on a token endpoint?

Solutions

1. The server keeps no memory of previous requests, so each request must carry everything needed to answer it; it makes conversational flows harder, since the client has to resend context.
2. The resource is the server-side thing; the representation is the document the client receives, and several representations can exist for one resource.
3. Benefit: repeated reads never touch your database. Disable it for responses that depend on the caller, such as a profile or a quota counter.
4. Level 0: one endpoint, one verb, the action is in the body.
5. A token is a credential: if a cache stored it, someone else could replay it. `no-store` forbids keeping a copy anywhere.

2.6 Mini test

1. Which constraint says a client cannot tell a proxy from the real server? A) client-server B) statelessness C) layered system D) code on demand
2. What does a client receive in a REST interaction? A) the resource itself B) a representation of the resource C) a database handle D) a session ticket
3. True or false: HTTP status codes belong to level 0 of the Richardson maturity model.
4. Which header on a response tells a cache to keep no copy of it? A) `Vary` B) `Cache-Control: no-store` C) `Content-Length` D) `Accept`
5. A `GET /api/v1/bookmarks?limit=1` response contains no cookie. Why is that consistent with statelessness?
6. Which two things make the uniform interface valuable to a new client?
7. What is code on demand, and why do most JSON APIs skip it?

Answer key and explanations

1. C - the layered system constraint hides proxies, gateways and load balancers behind the same interface.
2. B - the client only ever handles representations such as JSON.
3. False - meaningful status codes are what moves an API from level 1 to level 2.
4. B - `no-store` tells every cache on the path not to keep a copy.
5. Because nothing about the conversation is stored: the request itself carries `limit=1`, so the answer can be produced again at any time, by any node.
6. The same verb always means the same thing, and the same path always names the same kind of thing, so the client can predict behaviour without reading server code.
7. Shipping executable code to the client, as browsers do with JavaScript. Most JSON APIs skip it because their clients are other servers or apps that already have their logic.

Chapter 3 - Designing resources and URIs

3.1 Summary

- A URI names one thing; pick a name you will still like in two years.
- Use nouns for resources and let the HTTP method express the action.
- Collections are plural and take plural paths: `/api/v1/bookmarks`.
- A single member is the collection plus an identifier: `/api/v1/bookmarks/bm_003`.
- Path segments describe containment; query parameters describe options.
- Keep nesting to two levels at most, and flatten anything deeper into a query parameter.
- Identifiers in the path are opaque to the client; never encode business data in them if you can avoid it.
- Use lower case and hyphens for multi-word segments, and avoid file extensions.
- One resource may have many URIs, but every URI should mean exactly one thing.
- Consistency beats elegance: a plain naming scheme used everywhere is easier to learn than a clever one used once.

3.2 Concepts

Nouns, not verbs

The path answers "what?", the method answers "do what?". Once you internalise that split, most naming arguments disappear:

Intent	URL	Method
List bookmarks	`/api/v1/bookmarks`	`GET`
Create a bookmark	`/api/v1/bookmarks`	`POST`

Intent	URL	Method
Read one bookmark	`/api/v1/bookmarks/bm_003`	`GET`
Replace one bookmark	`/api/v1/bookmarks/bm_003`	`PUT`
Change part of one	`/api/v1/bookmarks/bm_003`	`PATCH`
Delete one bookmark	`/api/v1/bookmarks/bm_003`	`DELETE`
Mark one as favourite	`/api/v1/bookmarks/bm_003/favorite`	`PUT` or `DELETE`

Compare that with `/api/v1/getBookmarks`, `/api/v1/createBookmark`, `/api/v1/deleteBookmark?id=3`: three unrelated-looking paths that all use `GET` or `POST`, which no cache, gateway or log can reason about.

Plurals, singulars and identifiers

output

```
 collection                 member                      sub resource
/api/v1/bookmarks   -->   /api/v1/bookmarks/bm_003  -->  .../bm_003/tags? no

 12 items, sortable         one item, addressable       flatten into a field
```

Pick one convention and apply it to every collection. A mixed API where `/users` is plural but `/setting` is singular forces every client to remember exceptions.

When to nest, and when to stop

Nesting is containment, not decoration. `/users/7/bookmarks` is meaningful if a bookmark really only exists inside a user. `/users/7/bookmarks/3/tags/2/notes` is not: it is four levels of path and no additional meaning.

Depth	Example	Verdict
0	`/api/v1/bookmarks`	collection, fine
1	`/api/v1/bookmarks/bm_003`	member, fine
2	`/api/v1/users/u1/bookmarks`	contained collection, acceptable
3	`/api/v1/users/u1/bookmarks/bm_003/tags`	too deep, flatten it
3	`/api/v1/bookmarks/bm_003/tags`	borderline, prefer a field on the parent

A good escape hatch is a query parameter: instead of `/users/u1/bookmarks/bm_003/tags`, expose `?tag=http` on the collection, or return `tags` inside the bookmark representation.

Naming rules that never go out of style

Rule	Good	Bad
Lower case segments	`/api/v1/bookmarks`	`/api/v1/Bookmarks`
Hyphens for words	`/api/v1/api-keys`	`/api/v1/api_keys`, `/apiKeys`

Rule	Good	Bad
No trailing slash on members	`/api/v1/bookmarks/bm_003`	`/api/v1/bookmarks/bm_003/`
No file extensions	`/api/v1/bookmarks` (negotiate with `Accept`)	`/api/v1/bookmarks.json`
Stable ids only	`/api/v1/bookmarks/bm_003`	`/api/v1/bookmarks/mdn-http`
Options in the query string	`?sort=title`	`/sort/title`
Version first, resource second	`/api/v1/bookmarks`	`/bookmarks/v1`

3.3 Examples

Example 1: the route table of the demo API.

output

```
GET    /api/v1/health
POST   /api/v1/tokens
GET    /api/v1/me
GET    /api/v1/bookmarks
POST   /api/v1/bookmarks
GET    /api/v1/bookmarks/{bookmarkId}
PATCH  /api/v1/bookmarks/{bookmarkId}
DELETE /api/v1/bookmarks/{bookmarkId}
OPTIONS /api/v1/bookmarks        (CORS preflight, no body)
```

Eight routes, one resource family, no action names anywhere.

Example 2: a well formed collection URL with true output.

shell

```
curl.exe -s -i "http://127.0.0.1:3000/api/v1/bookmarks?limit=2"
```

HTTP

```
HTTP/1.1 200 OK
X-Request-Id: 62e30651-2ad5-40a8-8e00-9222f0231465
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 98
X-RateLimit-Reset: 1791669942
Content-Type: application/json; charset=utf-8
Content-Length: 792
Link: <http://127.0.0.1:3000/api/v1/bookmarks?limit=2&cursor=eyJvZmZzZXQiOjJ9>; rel="next"
Date: Sat, 10 Oct 2026 22:04:41 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "data": [
    {
      "id": "bm_001",
      "url": "https://developer.mozilla.org/en-US/docs/Web/HTTP",
      "title": "MDN HTTP reference",
      "description": "Protocol, methods, headers, status codes",
      "tags": [
        "http",
        "mdn"
      ],
      "favorite": true,
      "createdAt": "2026-09-01T09:00:00.000Z"
    },
    {
      "id": "bm_002",
      "url": "https://spec.openapis.org/oas/latest.html",
      "title": "OpenAPI Specification",
      "description": "The document format for REST APIs",
      "tags": [
        "openapi",
        "docs"
      ],
      "favorite": true,
      "createdAt": "2026-09-02T10:15:00.000Z"
    }
  ],
  "meta": {
    "limit": 2,
    "offset": 0,
    "total": 12,
    "hasMore": true,
    "nextCursor": "eyJvZmZzZXQiOjJ9"
  }
}
```

Example 3: a path that goes one level too deep.

shell

```
curl.exe -s -i http://127.0.0.1:3000/api/v1/bookmarks/bm_002/notes
```

HTTP

```
HTTP/1.1 404 Not Found
X-Request-Id: 4233fe08-2eb1-4826-ba9b-5532aab383e3
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 97
X-RateLimit-Reset: 1791670003
Content-Type: application/problem+json; charset=utf-8
Content-Length: 219
Date: Sat, 10 Oct 2026 22:05:43 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "type": "https://learnfree.dev/problems/not-found",
  "title": "Route not found",
  "detail": "no route matches GET /api/v1/bookmarks/bm_002/notes",
  "instance": "/api/v1/bookmarks/bm_002/notes"
}
```

The demo API deliberately has no third level, so a client that invents one gets an honest `404` instead of a silently empty list. Prefer `?tag=http` on the collection over inventing sub-paths for every attribute.

Example 4: build a URL without string concatenation.

JavaScript

```
const base = new URL("http://127.0.0.1:3000/api/v1/bookmarks");
base.searchParams.set("tag", "mdn");
base.searchParams.set("sort", "title");
base.searchParams.set("order", "desc");
base.searchParams.set("limit", "2");
console.log(base.toString());
const shown = [...base.searchParams].map((p) => p.join("="));
console.log(base.pathname + " ->", shown.join("&"));
```

output

```
http://127.0.0.1:3000/api/v1/bookmarks?tag=mdn&sort=title&order=desc&limit=2
/api/v1/bookmarks -> tag=mdn&sort=title&order=desc&limit=2
```

`URLSearchParams` escapes values for you. Building URLs with `+` and template strings is how broken encoding and double question marks enter a codebase.

Example 5: identifiers are opaque.

JavaScript

```
const id = "bm_003";
const path = `/api/v1/bookmarks/${id}`;
console.log(path);
console.log("encoded  :", encodeURIComponent("/api/v1/bookmarks/bm 003"));
console.log("safely id:", `/api/v1/bookmarks/${encodeURIComponent("bm 003")}`);
```

output

```
/api/v1/bookmarks/bm_003
encoded  : %2Fapi%2Fv1%2Fbookmarks%2Fbm%20003
safely id: /api/v1/bookmarks/bm%20003
```

Never splice a user supplied value into a path without encoding it, and treat the id as a token: the day you move from `bm_003` to a UUID, clients that parsed the old format break.

3.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Verbs in the path	the method and the URL disagree	nouns in the path, verbs in the method
Mixed singular and plural collections	clients must memorise exceptions	always plural
Deep nesting for attributes	unreadable URLs, useless joins	expose a query parameter or a field
Meaningful, guessable ids	they leak information and get renamed	opaque ids, stable over time
`.json` or `.xml` in the path	the format is negotiated, not named	use the `Accept` header
Query strings that change state	caches and prefetchers repeat them	state changes belong to `POST`, `PATCH`, `DELETE`
Trailing slashes that matter	two URLs, one resource	redirect or accept both, but pick one

- Write the route table first, then the handlers; it is the cheapest design review you will ever do.
- If two URLs would return the same thing for the same client, merge them.

3.5 Practice exercises

1. Rewrite `GET /api/v1/getUserBookmarks?userId=7` using nouns and the right method.
2. Why is `/api/v1/bookmarks/bm_003/tags/http/delete` a bad design?
3. Give a better URL for `/api/v1/users/7/bookmarks/bm_003/tags/http`.
4. Which is better, `/api/v1/apiKeys` or `/api/v1/api-keys`, and why?
5. A client sends `GET /api/v1/bookmarks/bm_003/`. What should the server do?

Solutions

1. `GET /api/v1/users/7/bookmarks` - the resource is the collection of one user's bookmarks, and `GET` already says "read".
2. It mixes a verb into the path, nests three levels deep, and puts an attribute in the URL; the collection plus a `?tag=http` filter is enough.

3. `GET /api/v1/users/7/bookmarks?tag=http`, reading the tag as a filter on the contained collection.
4. `/api/v1/api-keys`: lower case with hyphens is the convention that survives proxies, logs and case sensitive routers.
5. Either answer normally or redirect to the canonical path without the slash; what matters is that both spellings never return different documents.

3.6 Mini test

1. Which part of the request expresses the action? A) the path B) the query string C) the method D) the header
2. Which path is best for one bookmark? A) `/api/v1/bookmark?id=bm_003` B) `/api/v1/bookmarks/bm_003` C) `/api/v1/getBookmark/bm_003` D) `/api/v1/bookmarks/bm_003.json`
3. True or false: nesting depth in a URL should reflect database table joins.
4. How deep should nesting normally go? A) at most 2 levels below the collection B) as deep as the schema requires C) exactly 3 D) nesting is forbidden
5. Where do sorting and filtering options belong? A) in the path B) in the query string C) in a header D) in the body of a `GET`
6. Why are ids such as `bm_003` preferred over `mdn-http-reference`?
7. Name the header used to negotiate the response format.

Answer key and explanations

1. C - the method carries the intent; the path names the resource.
2. B - plural collection, member id, no extension and no verb.
3. False - the URL describes the API's resource model, not the internal schema; joins are an implementation detail.
4. A - two levels keep URLs readable and leave room for filters.
5. B - query parameters adjust a representation without changing identity.
6. Opaque ids survive renames, sorting changes and migrations; a meaningful id invites clients to parse it and breaks the day it changes.
7. `Accept`, for example `Accept: application/json`.

Chapter 4 - HTTP methods done right

4.1 Summary

- Methods carry intent: `GET` reads, `POST` creates, `PUT` replaces, `PATCH` modifies, `DELETE` removes.
- A safe method never changes server state; `GET`, `HEAD` and `OPTIONS` are safe, the others are not.
- An idempotent method produces the same server state no matter how often it is repeated.
- `GET`, `HEAD`, `PUT`, `DELETE` and `OPTIONS` are idempotent; `POST` is the one that is not.

- `PUT` replaces an entire representation, `PATCH` changes only the fields you send.
- Because `POST` is not idempotent, a retried POST can create duplicates, which is what `Idempotency-Key` headers are for.
- An unsupported method on a known path must answer `405` with an `Allow` header.
- Unknown paths answer `404`, and the two must never be confused.
- Never put a state change behind `GET`: browsers prefetch, caches replay and crawlers follow links.
- When you choose a method, ask what happens if the request is sent twice.

4.2 Concepts

Safe and idempotent

Method	Safe	Idempotent	Has body	Typical statuses
`GET`	yes	yes	no	`200`, `404`, `400`
`HEAD`	yes	yes	no	`200`, `404`
`OPTIONS`	yes	yes	optional	`204`, `200`
`POST`	no	no	yes	`201`, `200`, `400`, `409`
`PUT`	no	yes	yes	`200`, `201`, `400`
`PATCH`	no	yes	yes	`200`, `400`, `404`
`DELETE`	no	yes	optional	`204`, `200`, `404`

Safety means "no side effects", which is what makes `GET` cacheable and safe to prefetch. Idempotence means "repeating changes nothing further", which is what lets a client retry after a timeout without asking the user.

output

```
retry safety

GET    /bookmarks         1000 times -> 1000 identical reads
PUT    /bookmarks/bm_003  1000 times -> the same replacement, stored once
PATCH  /bookmarks/bm_003  1000 times -> favorite stays true
DELETE /bookmarks/bm_003  1000 times -> gone after the first, 404 after that
POST   /bookmarks         1000 times -> 1000 new bookmarks   (careful!)
```

`PUT` against `PATCH`

Question	PUT	PATCH
What does the body contain?	the whole resource	only the fields to change
What happens to omitted fields?	they are reset or the request is rejected	they keep their current value

Question	PUT	PATCH
Is it idempotent?	yes	yes, for a well designed patch format
When is it right?	full syncs, file-like uploads	forms and partial edits

A `PATCH` body of `{"favorite":true}` must not blank out the title. A `PUT` body of `{"favorite":true}` either wipes the rest of the resource or is rejected as incomplete. Mixing the two is one of the most common sources of data loss in public APIs.

`POST`, retries and the idempotency key

`POST /api/v1/bookmarks` sent twice creates two bookmarks. If the client never learned whether the first request arrived, a retry is a coin flip. The standard answer is an `Idempotency-Key` header: the client generates a unique value, the server remembers the result for a while, and a repeat with the same key returns the stored response instead of doing the work again.

HTTP

```http
POST /api/v1/bookmarks HTTP/1.1
Content-Type: application/json
Idempotency-Key: 6f1c2b7e-1d4a-4a1f-9a2e-2c4b8f0d1111

{"title":"Node.js docs","url":"https://nodejs.org/docs/latest/api/"}
```

The demo API does not implement this header, because the point of this volume is the design rules you can apply to an API that does.

`HEAD` and `OPTIONS`

`HEAD` is a `GET` without a body, useful for checking `Content-Length`, `ETag` or `Last-Modified` before downloading. `OPTIONS` asks what a resource allows, and it is also how browsers perform a CORS preflight, which chapter 11 covers in detail.

4.3 Examples

Example 1: one resource, five methods, five answers.

shell

```shell
B=http://127.0.0.1:3000/api/v1/bookmarks/bm_003
code() { curl.exe -s -o NUL -w '%{http_code}\n' "$@"; }
code $B
code -X POST $B
code -X PATCH $B
code -X DELETE $B
```

output

```
200
405
401
401
```

The member path accepts `GET`, `PATCH` and `DELETE`, so `POST` is refused with `405`. Writes need credentials, which is why `PATCH` and `DELETE` answer `401` until you send the `X-API-Key` header

shown in example 3.

Example 2: `HEAD` on a collection that does not support it.

shell

```shell
curl.exe -s -I "http://127.0.0.1:3000/api/v1/bookmarks?limit=1"
```

HTTP

```http
HTTP/1.1 405 Method Not Allowed
X-Request-Id: 95d12ca7-51d5-47f6-b401-9bf7e4db61788f1
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 94
X-RateLimit-Reset: 1791670360
Content-Type: application/problem+json; charset=utf-8
Content-Length: 209
Allow: GET, POST, OPTIONS
```

The demo server keeps its routes deliberately small, so `HEAD` is not implemented. Notice the `Allow` header: a client reading it learns, without guesswork, that `GET`, `POST` and `OPTIONS` are the options. A production API should support `HEAD` on every `GET` route.

Example 3: a partial update with `PATCH`.

shell

```shell
curl.exe -s -i -X PATCH http://127.0.0.1:3000/api/v1/bookmarks/bm_013 \
  -H "Content-Type: application/json" -H "X-API-Key: lf-demo-key-123" \
  --data "@bodies/patch.json"
```

HTTP

```
HTTP/1.1 200 OK
X-Request-Id: e66815e9-4188-4e6f-8cde-8e587257cfc7
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 95
X-RateLimit-Reset: 1791669960
Content-Type: application/json; charset=utf-8
Content-Length: 254
Date: Sat, 10 Oct 2026 22:05:06 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "id": "bm_013",
  "url": "https://nodejs.org/docs/latest/api/",
  "title": "Node.js docs",
  "description": "The official runtime documentation",
  "tags": [
    "node",
    "docs"
  ],
  "favorite": true,
  "createdAt": "2026-10-10T22:05:06.158Z"
}
```

The body sent was `{"favorite":true}`. Everything else survived: the title, the url, the tags and the original `createdAt` are untouched. That is the whole contract of `PATCH` in one response.

Example 4: `DELETE` is idempotent, not magic.

shell

```
D() { curl.exe -s -o NUL -w '%{http_code}\n' "$@"; }
D -X DELETE http://127.0.0.1:3000/api/v1/bookmarks/bm_013 \
  -H "X-API-Key: lf-demo-key-123"
D -X DELETE http://127.0.0.1:3000/api/v1/bookmarks/bm_013 \
  -H "X-API-Key: lf-demo-key-123"
```

output

```
204
404
```

The first call deletes the bookmark and answers `204 No Content`. The second call has nothing to delete, so it answers `404`. The server state after the second call is identical to the state after the first, which is exactly what idempotence means. A client that retries after a lost connection is not corrupting anything.

Example 5: a `PUT` that the API does not implement.

shell

```
curl.exe -s -i -X PUT http://127.0.0.1:3000/api/v1/bookmarks/bm_003 \
  -H "X-API-Key: lf-demo-key-123"
```

HTTP

```
HTTP/1.1 405 Method Not Allowed
X-Request-Id: 42905adf-e743-42f2-91d3-4c8ab561f5b1
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 95
X-RateLimit-Reset: 1791670360
Content-Type: application/problem+json; charset=utf-8
Content-Length: 209
Allow: GET, PATCH, DELETE, OPTIONS
Date: Sat, 10 Oct 2026 22:11:40 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "type": "https://learnfree.dev/problems/method-not-allowed",
  "title": "Method not allowed",
  "status": 405,
  "detail": "PUT is not supported on /api/v1/bookmarks/bm_003",
  "instance": "/api/v1/bookmarks/bm_003"
}
```

Refusing a method you do not want to support is healthier than half-implementing it. If a client sends `PUT`, it expects a full replacement; answering `405` with the `Allow` list is an honest "use `PATCH` instead".

4.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`GET` with side effects	prefetches and caches repeat them	state changes use `POST`, `PATCH`, `PUT` or `DELETE`
Using `POST` for updates	the URL stops identifying the resource	`PUT` or `PATCH` on the member path
Accepting `PATCH` bodies that blank fields	silent data loss	merge, never replace, and validate
Implementing `PUT` as a partial update	clients cannot tell the difference	pick one meaning per method and document it
Retrying `POST` blindly	duplicates	send an `Idempotency-Key` and honour it
Answering `404` for an unsupported method	the client thinks the resource is gone	`405` plus an `Allow` header
Long request bodies on `GET`	proxies drop them	read-only requests carry everything in the URL

- Write down, for every route, what happens when the request is repeated twice.
- If your method choice is hard to explain in one sentence, the resource model is wrong.

4.5 Practice exercises

1. Which methods are both safe and idempotent, and why does that combination matter?
2. A client sends `PATCH {"title":""}`. What should the server do and why?
3. Why is `DELETE` idempotent even though it changes state?
4. What is the difference between `404` and `405`, and which extra header belongs on a `405`?
5. How does an `Idempotency-Key` header make a `POST` safe to retry?

Solutions

1. `GET`, `HEAD` and `OPTIONS`. It matters because clients, caches, crawlers and browsers may repeat them at any time without harming anything.
2. Reject it with `400` and a validation error naming the `title` field: an empty title is not a meaningful value, and accepting it would corrupt the resource.
3. Repeating it after the first success leaves the same state: the resource is gone. Only the status code changes (`404`), never the stored data.
4. `404` means nothing matches that path at all; `405` means the path exists but not for that method. A `405` carries an `Allow` header listing the supported methods.
5. The client sends a unique key, the server stores the outcome of the first request against that key, and a repeat returns the stored response instead of running the handler again.

4.6 Mini test

1. Which method is neither safe nor idempotent? A) `GET` B) `DELETE` C) `POST` D) `PUT`
2. What does a `PUT` body contain? A) the fields you want to change B) the complete representation C) only the resource id D) a list of operations
3. Which status answers a method that a known path does not support? A) `400` B) `404` C) `405` D) `501`
4. True or false: `DELETE` on a missing resource should answer `204` to keep the method idempotent.
5. A `PATCH` sends `{"favorite":true}`. What must happen to the other fields?
6. Which header must a browser test before sending `DELETE` with a JSON body?
7. Why is `GET /api/v1/bookmarks?limit=1` a good request and `GET /api/v1/deleteBookmark/3` a bad one?

Answer key and explanations

1. C - `POST` creates something new each time, so repeating it repeats the effect.
2. B - `PUT` replaces the whole representation; anything omitted is gone.
3. C - the path is valid, the verb is not; `405` with `Allow` tells the client what is possible.
4. False - the stored state is the same either way, so honesty wins: `404` tells the client the resource is gone. Some APIs answer `204` for convenience, but that hides a real condition.
5. They keep their current values; a patch only touches what it names.

6. The CORS preflight, answered by `OPTIONS`: a cross-origin request with a JSON body is not a "simple" request, so the browser asks permission first.
7. The first names a resource and filters it; the second hides an action in the path and uses `GET` to change data.

Chapter 5 - Status codes and error payloads

5.1 Summary

- Status codes come in five families: informational, success, redirection, client error and server error.
- Pick the code that describes the outcome, and keep the body for the details.
- `200`, `201`, `204` and `202` are the success codes a JSON API actually needs.
- `400`, `401`, `403`, `404`, `405`, `409`, `415` and `422` cover almost every client mistake.
- `500` and `503` are the only server errors you should send on purpose.
- Every error body should have the same shape, so clients can parse it once.
- RFC 9457 defines `application/problem+json` with `type`, `title`, `status` and `detail`.
- A machine readable `type` URI beats a human readable message when it comes to branching in code.
- Validation failures should list every bad field, not just the first one.
- Never send `200` with an error inside the body: caches, proxies and monitoring all read the status line first.

5.2 Concepts

The families

Class	Meaning	Typical use
1xx	informational	rare in APIs; `100 Continue` while a body is uploaded
2xx	success	the request did what it asked
3xx	redirection	the resource moved, or the canonical URL is elsewhere
4xx	client error	the request was wrong or not allowed
5xx	server error	the request was fine, the server failed

The shortlist worth memorising

Code	Name	Use it when
200	OK	a read or an update worked

Code	Name	Use it when
`201`	Created	a `POST` created a resource; send `Location`
`202`	Accepted	the work was queued and will finish later
`204`	No Content	a `DELETE` worked; there is no body
`304`	Not Modified	a conditional `GET` found nothing new
`400`	Bad Request	the body or query is malformed or invalid
`401`	Unauthorized	no credential, or an invalid one
`403`	Forbidden	the credential is valid but not allowed
`404`	Not Found	nothing matches that path
`405`	Method Not Allowed	the path exists, the verb does not
`409`	Conflict	the request collides with existing state
`412`	Precondition Failed	an `If-Match` guard did not hold
`415`	Unsupported Media Type	the body format is not accepted
`422`	Unprocessable Content	syntactically valid but semantically wrong
`429`	Too Many Requests	the client exhausted its quota
`500`	Internal Server Error	an unhandled bug
`503`	Service Unavailable	the server is up but refusing work

`401` and `403` are the pair people mix up. `401` means "I do not know who you are"; `403` means "I know who you are and the answer is no".

One error shape for everything

RFC 9457, Problem Details for HTTP APIs, standardises an error document. In its simplest form it has four fields:

Field	Meaning
`type`	a URI identifying the kind of problem
`title`	a short, human readable summary
`status`	the HTTP status code, repeated for convenience
`detail`	a sentence about this specific occurrence

The demo API adds two optional fields of its own: `instance`, the path the client asked for, and `errors`, a list of field level problems. Extensions are explicitly allowed, which is what makes the format usable for validation.

output

```
  client reads "type"      -> branches on the machine readable class
  client reads "detail"    -> shows or logs the sentence
  client reads "errors"    -> highlights the form fields

  { "type": ".../validation-error", "status": 400,
    "title": "Request body failed validation",
    "detail": "1 field is invalid",
    "instance": "/api/v1/bookmarks",
    "errors": [ { "field": "title", "code": "required" } ] }
```

Every error response in the demo API also carries `Content-Type: application/problem+json`, so a client can dispatch on the content type before it even reads the status code.

5.3 Examples

Example 1: a problem document for a missing resource.

shell

```
curl.exe -s -i http://127.0.0.1:3000/api/v1/bookmarks/bm_999
```

HTTP

```
HTTP/1.1 404 Not Found
X-Request-Id: 416ed1f0-305c-4f0f-b9b8-a992f6e30aa1
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 96
X-RateLimit-Reset: 1791669942
Content-Type: application/problem+json; charset=utf-8
Content-Length: 198
Date: Sat, 10 Oct 2026 22:04:41 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "type": "https://learnfree.dev/problems/not-found",
  "title": "Bookmark not found",
  "status": 404,
  "detail": "no bookmark has the id \"bm_999\"",
  "instance": "/api/v1/bookmarks/bm_999"
}
```

Example 2: a validation failure that lists every bad field.

shell

```
curl.exe -s -i -X POST http://127.0.0.1:3000/api/v1/bookmarks \
  -H "Content-Type: application/json" -H "X-API-Key: lf-demo-key-123" \
  --data "@bodies/missing-title.json"
```

HTTP

```
HTTP/1.1 400 Bad Request
X-Request-Id: 7b0a2e48-14e0-458f-8181-5d26465b904b
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 98
X-RateLimit-Reset: 1791669960
Content-Type: application/problem+json; charset=utf-8
Content-Length: 325
Date: Sat, 10 Oct 2026 22:04:59 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "type": "https://learnfree.dev/problems/validation-error",
  "title": "Request body failed validation",
  "status": 400,
  "detail": "1 field is invalid",
  "instance": "/api/v1/bookmarks",
  "errors": [
    {
      "field": "title",
      "code": "required",
      "message": "a non empty title is required"
    }
  ]
}
```

Each entry in `errors` names the field, gives a short machine readable `code` and a sentence you can show a user. A client can highlight the offending input without parsing English.

Example 3: authentication and authorisation failures.

shell

```
curl.exe -s -i -X POST http://127.0.0.1:3000/api/v1/bookmarks \
  -H "Content-Type: application/json" --data "@bodies/duplicate.json"
```

HTTP

```
HTTP/1.1 401 Unauthorized
X-Request-Id: 082153d4-0b6e-44aa-abd5-27940b56666f
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 89
X-RateLimit-Reset: 1791669942
Content-Type: application/problem+json; charset=utf-8
Content-Length: 175
WWW-Authenticate: ApiKey realm=bookmarks
Date: Sat, 10 Oct 2026 22:04:58 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "type": "https://learnfree.dev/problems/invalid-api-key",
  "title": "Invalid API key",
  "status": 401,
  "detail": "Write operations require a valid X-API-Key header"
}
```

`WWW-Authenticate` is the standard way to say which credential scheme to use. Reading it is what lets a client library ask for a key instead of guessing.

Example 4: conflicts and unsupported media types.

shell

```
curl.exe -s -i -X POST http://127.0.0.1:3000/api/v1/bookmarks \
  -H "Content-Type: application/json" -H "X-API-Key: lf-demo-key-123" \
  --data "@bodies/duplicate.json"
```

output

```
HTTP/1.1 409 Conflict

{
  "type": "https://learnfree.dev/problems/duplicate-url",
  "title": "Bookmark already exists",
  "status": 409,
  "detail": "a bookmark with this URL is already stored",
  "instance": "/api/v1/bookmarks"
}
```

The same body sent with `Content-Type: text/plain` answers `415 Unsupported Media Type` with the detail `expected application/json, received "text/plain"`, which is far more useful than a generic `400`.

Example 5: server errors stay boring.

JavaScript

```
function handle(request, response) {
  try {
    const page = readBookmarks(request);
    response.writeHead(200, { "Content-Type": "application/json" });
    response.end(JSON.stringify(page));
  } catch (error) {
    console.error(error);
    response.writeHead(500, { "Content-Type": "application/problem+json" });
    response.end(JSON.stringify({
      type: "https://learnfree.dev/problems/internal-error",
      title: "Internal error",
      status: 500,
      detail: "the request could not be completed",
    }));
  }
}
```

output

```
(no output; a failing request produces a 500 problem document and a stack
trace in the server log, never in the response body)
```

Log the stack trace on the server, send a stable document to the client. A `500` that echoes an exception is a free map of your codebase for an attacker.

5.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`200` with an error in the body	caches and monitoring are fooled	the status line carries the outcome
`500` for a bad request	the client retries something that will never work	`4xx` for the caller's mistakes, `5xx` for yours
A different error shape per route	every client needs a parser per route	one problem document everywhere
Only the first validation error returned	users fix one field, resubmit, find another	return every field error at once
Raw exception text in the response	leaks paths, queries and versions	log details, return a generic problem
Using `404` for "exists but not yours"	hides authorisation problems	`403`, or `404` only when you must hide existence
Inventing new status codes	proxies and clients ignore them	use the registry, extend with `type`

- Keep a single table of the codes your API can return and maintain it like documentation.
- When you cannot pick a code, ask whether the client can fix the problem: if yes, `4xx`; if not, `5xx`.

5.5 Practice exercises

1. Which status code should a `POST` return after creating a resource, and which header goes with it?
2. What is the difference between `401` and `403`?
3. Name the four required fields of an RFC 9457 problem document.
4. A client sends a JSON body with `title: 42`. Which status code, and why?
5. Why is `503` a better answer than `500` during a planned maintenance window?

Solutions

1. `201 Created`, with a `Location` header pointing at the new resource, for example `/api/v1/bookmarks/bm_013`.
2. `401` means the credential is missing or invalid; `403` means the caller is known but is not allowed to do this.
3. `type`, `title`, `status` and `detail`.
4. `400` (or `422`): the JSON parses, but the value is not a string, so the request is not processable. Report the field in `errors`.
5. `503` says "the service is temporarily unavailable", which invites a retry after a delay, and it pairs naturally with `Retry-After`.

5.6 Mini test

1. Which status code means "created"? A) `200` B) `201` C) `202` D) `204`
2. Which class of codes means the client sent something wrong? A) `2xx` B) `3xx` C) `4xx` D) `5xx`
3. Which media type does RFC 9457 define for problem documents? A) `application/json` B) `application/problem+json` C) `text/plain` D) `application/problem+xml`
4. True or false: sending a stack trace inside a `500` response is a good debugging aid for clients.
5. A `DELETE` succeeds and there is nothing to return. Which status code fits best?
6. Which status code tells the caller their quota is exhausted?
7. Why should a validation error list every invalid field at once?

Answer key and explanations

1. B - `201 Created` marks a new resource; add a `Location` header.
2. C - `4xx` reports a problem with the request itself.
3. B - `application/problem+json` is the registered media type for problem details.
4. False - it leaks internals; log the trace on the server and return a stable problem document.
5. `204 No Content`, because the work is done and there is no representation to send back.
6. `429 Too Many Requests`, ideally with `Retry-After` and rate limit headers.
7. Because each round trip costs the user time; reporting everything at once lets them fix the whole form in one go.

Chapter 6 - Requests, responses and JSON hygiene

6.1 Summary

- A JSON API speaks two dialects: the request dialect the client sends and the response dialect it expects back.
- `Content-Type` describes the body you are sending; `Accept` describes the body you want back.
- Headers are metadata; keep the data itself in the body, not in custom headers.
- Wrap collections in an envelope with the data and a `meta` block describing the page.
- Field names follow one convention from end to end; `camelCase` is the common choice for JSON APIs.
- Use closed sets of values (`enum`s) for anything a client must branch on.
- Represent "nothing" deliberately: `null` for a known empty value, an absent key for "not provided".
- Timestamps are strings in ISO 8601 with a UTC offset, never local time and never a bare number.
- Booleans are `true` and `false`, not `"true"`, `1` or `"yes"`.
- A response schema is a promise: once clients depend on a field, removing it is a breaking change.

6.2 Concepts

The two sides of the exchange

Concern	Request side	Response side
Format	`Content-Type: application/json`	`Content-Type: application/json; charset=utf-8`
Preference	`Accept: application/json`	whatever the server chose
Size	`Content-Length`	`Content-Length`
Identity	`X-API-Key` or `Authorization`	`X-Request-Id`
Policy	-	`X-RateLimit-*`, `Retry-After`, `Cache-Control`
Negotiation result	-	`Vary: Origin` announces that the answer depends on the request

`Vary` matters more than it looks: it tells a cache that two requests that look similar may get different answers, which is what keeps a shared cache from handing the wrong document to the wrong origin.

Naming and typing rules

Rule	Do this	Not this
Case	`createdAt`, `apiKey`	`created_at`, `api_key`, `CreatedAt`
Booleans	`favorite: true`	`favorite: "true"`, `favorite: 1`
Closed sets	`status: "active"`	`status: 1`, `isActive: "Y"`
Money and counts	numbers	`"12"`, `"$12.00"`
Time	`"2026-09-03T08:40:00.000Z"`	`"03/09/2026"`, `1756879200`
Collections	arrays	objects keyed by position (`"0": ...`)
Identifiers	`"bm_003"`	`3` (a number invites arithmetic)
Empty list	`[]`	`null`, `""`

`null` versus an absent key

output

```
{"description": null}    the field exists and its value is nothing
{}                       the field was not part of this representation

a PATCH of {"description": null}   clears the description
a PATCH of {}                      leaves the description alone
```

The distinction is what makes partial updates work. If a client cannot tell the two apart, every patch must resend the whole object, which is `PUT` in disguise.

The envelope

A collection answer needs more than the rows: how many there are, how many you got, and where the next page lives.

output

```
{
  "data":  [ ...the resources... ],
  "meta":  { "limit": 2, "offset": 0, "total": 12,
             "hasMore": true, "nextCursor": "eyJvZmZzZXQiOjJ9" }
}
```

Some teams prefer `items` plus `pageInfo`; others follow the structure that `jsonapi.org` documents. Any of them is fine as long as it never changes.

6.3 Examples

Example 1: what the server sends when the client asks for the wrong format.

shell

```
curl.exe -s -i -H "Accept: text/plain" http://127.0.0.1:3000/api/v1/bookmarks/bm_001
```

HTTP

```
HTTP/1.1 200 OK
X-Request-Id: 126ce5d7-3784-408d-84ac-0e8546c310c7
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 99
X-RateLimit-Reset: 1791670598
Content-Type: application/json; charset=utf-8
Content-Length: 279
Date: Sat, 10 Oct 2026 22:15:37 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "id": "bm_001",
  "url": "https://developer.mozilla.org/en-US/docs/Web/HTTP",
  "title": "MDN HTTP reference",
  "description": "Protocol, methods, headers, status codes",
  "tags": [
    "http",
    "mdn"
  ],
  "favorite": true,
  "createdAt": "2026-09-01T09:00:00.000Z"
}
```

The demo API only speaks JSON, so it answers with `application/json` even though the client asked for `text/plain`. A production API with several formats should answer `406 Not Acceptable` when it cannot satisfy `Accept`.

Example 2: one field name, one convention.

JavaScript

```
const snake = { bookmark_id: "bm_003", created_at: "2026-09-03T08:40:00.000Z",
                is_favorite: false, tag_list: ["http", "rfc"] };
const camel = { bookmarkId: "bm_003", createdAt: "2026-09-03T08:40:00.000Z",
                favorite: false, tags: ["http", "rfc"] };
console.log(Object.keys(snake).join(", "));
console.log(Object.keys(camel).join(", "));
console.log("favorite in camel is a", typeof camel.favorite);
```

output

```
bookmark_id, created_at, is_favorite, tag_list
bookmarkId, createdAt, favorite, tags
favorite in camel is a boolean
```

`is_favorite` and `favorite` bake the type into the name twice: once as a prefix and once as a boolean value. Pick one convention and let the type speak for itself.

Example 3: `null`, absent and `undefined` on the wire.

JavaScript

```
const explicit = { title: "MDN HTTP reference", description: null, tags: [] };
const minimal = { title: "MDN HTTP reference", tags: [] };
const dropped = { title: "MDN HTTP reference", description: undefined };
console.log(JSON.stringify(explicit));
console.log(JSON.stringify(minimal));
console.log(JSON.stringify(dropped));
console.log("keys in dropped:", Object.keys(dropped).join(", "));
```

output

```
{"title":"MDN HTTP reference","description":null,"tags":[]}
{"title":"MDN HTTP reference","tags":[]}
{"title":"MDN HTTP reference"}
keys in dropped: title
```

`JSON.stringify` drops a key whose value is `undefined`, so a field you meant to clear disappears instead of becoming `null`. Set it explicitly when clearing.

Example 4: timestamps that survive a timezone change.

JavaScript

```
const createdAt = new Date("2026-09-03T08:40:00.000Z");
console.log(createdAt.toISOString());
console.log(Date.parse("2026-09-03T08:40:00.000Z"));
console.log(new Date(Date.parse("2026-09-03T08:40:00.000Z")).toISOString());
```

output

```
2026-09-03T08:40:00.000Z
1788424800000
2026-09-03T08:40:00.000Z
```

ISO 8601 with an explicit `Z` is unambiguous, sorts alphabetically, and can be parsed by every language without knowing the server's locale.

Example 5: closed sets of values.

JavaScript

```
const FILTERS = ["all", "unread", "archived"];
const filter = "unread";
console.log("accepted:", FILTERS.includes(filter));
console.log("accepted:", FILTERS.includes("everything"));
```

output

```
accepted: true
accepted: false
```

Validating against a list of allowed values turns a typo into a `400` with a useful message, instead of an empty result set that looks like a bug.

6.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Mixing `camelCase` and `snake_case`	clients write adapters	one convention, stated in the docs
Booleans as strings	`if (favorite)` is always true	real `true` and `false`
Local time or formatted dates in the API	timezone bugs everywhere	ISO 8601 with an offset
Numbers as strings	arithmetic breaks in strict clients	use JSON numbers
Removing a field because nobody uses it yet	someone does, and their app breaks	deprecate first, remove in a new version
Business data in custom headers	invisible to most tooling	keep it in the body
Different envelopes per route	every client grows special cases	one envelope, one error shape
`null` lists	every consumer needs a guard	return `[]`

- Write the response schema before the handler; if you cannot draw it, you cannot document it.
- When you must break a field, add the new one beside the old one and remove the old one in the next version.

6.5 Practice exercises

1. What is the difference between `Content-Type` and `Accept`?
2. Why is `"favorite": "true"` worse than `"favorite": true`?
3. What does `null` mean in a response, and how is it different from an absent key?
4. Which timestamp format should a JSON API use, and why?
5. What goes in the `meta` block of a collection response?

Solutions

1. `Content-Type` describes the body being sent; `Accept` lists the formats the client is willing to receive.
2. It is truthy in almost every language, so `if (bookmark.favorite)` is always true and the type cannot be trusted by a schema.
3. `null` means the field exists and its value is nothing; an absent key means the field was not part of this document at all, which is what lets a `PATCH` leave it untouched.
4. ISO 8601 with an explicit offset, such as `2026-09-03T08:40:00.000Z`: it is unambiguous, sortable and parseable everywhere.
5. The page description: how many items were requested and returned, how many exist in total, whether more pages follow, and the cursor or offset for the next one.

6.6 Mini test

1. Which header says what the client wants back? A) `Content-Type` B) `Accept` C) `Vary` D) `X-Request-Id`
2. Which value is a correct boolean in JSON? A) `"true"` B) `1` C) `true` D) `"yes"`
3. Which timestamp is best for an API? A) `03/09/2026 08:40` B) `2026-09-03 08:40 local` C) `2026-09-03T08:40:00.000Z` D) `1756879200`
4. True or false: an empty collection may be returned as `null`.
5. What does `Vary: Origin` tell a cache?
6. Why should identifiers be strings rather than numbers?
7. A client sends `PATCH {"description": null}`. What is the intended effect?

Answer key and explanations

1. B - `Accept` is the client's preference list; `Content-Type` describes the body being sent.
2. C - JSON has a real boolean type, so use it.
3. C - ISO 8601 with an explicit UTC offset.
4. False - return `[]`; `null` forces every consumer to write a guard.
5. That the response depends on the request's `Origin` header, so a shared cache must not reuse it for a different origin.
6. Numbers invite arithmetic, sorting assumptions and precision problems; identifiers are labels, not quantities.
7. Clearing the description: the key is present with an explicit "nothing", which is different from omitting the key.

Chapter 7 - Filtering, sorting and query design

7.1 Summary

- Query parameters are the API's control panel: filtering, sorting, shaping and paging.
- Filter by field name (`?tag=http`), search with a dedicated term (`?q=jwt`), and keep both.
- Sorting needs a field and a direction: `?sort=title&order=desc`.
- Validate every query parameter and answer `400` with a problem document that names the offender.
- Give every collection a default sort, and make it deterministic.
- A stable sort order is what makes cursor pagination trustworthy.
- Reserved parameter names become part of your contract the moment a client uses them.
- Do not let filters create, update or delete anything.
- Cap `limit`, and document the cap.

- Filters compose: `?tag=http&favorite=true&sort=title&order=asc&limit=2` should just work.

7.2 Concepts

The vocabulary of a collection endpoint

Parameter	Purpose	Example
`tag`	exact match on a field	`?tag=mdn`
`favorite`	exact match on a boolean	`?favorite=true`
`q`	free text search over several fields	`?q=jwt`
`sort`	field to order by	`?sort=createdAt`
`order`	direction, `asc` or `desc`	`?order=desc`
`limit`	page size	`?limit=2`
`offset`	items to skip	`?offset=6`
`cursor`	opaque position	`?cursor=eyJvZmZzZXQiOjZ9`
`fields`	sparse fieldsets	`?fields=id,title`

Filtering shapes

Shape	Syntax	Notes
Equality	`?status=active`	the default and the most common
Boolean	`?favorite=true`	accept only `true` and `false`
Multiple values	`?tag=http&tag=rfc`	means "any of", documented explicitly
Range	`?createdAfter=2026-09-01`	comparisons need a field specific name
Search	`?q=http+semantics`	one term, several fields, case insensitive
Negation	`?favorite=false`	cheaper and clearer than `?notFavorite=1`

Sorting and stability

Ordering a collection needs two decisions: which field, and which direction. Both belong in the query string, both are validated against a list of allowed fields, and the result must be deterministic.

output

```
?sort=createdAt&order=desc

bm_012  2026-09-16
bm_011  2026-09-15
bm_010  2026-09-14
bm_009  2026-09-12      <- ties are broken by a second key,
...                        usually the id, so the order never wobbles
```

Without a tie-breaker, two rows with the same value can swap places between requests. A cursor built on an unstable order then skips or repeats items.

Validation, not silent tolerance

A query parameter you do not understand is a bug in the client, and silently ignoring it produces results that look correct but are not. Validate instead:

Input	Answer
`?sort=colour`	`400` naming `sort` and the allowed values
`?limit=abc`	`400` naming `limit`
`?limit=1000`	clamp to the documented maximum, or `400`
`?cursor=nope`	`400` naming `cursor`
unknown parameter	ignore it, or reject it; pick one and document it

7.3 Examples

Example 1: filter and sort together.

shell

```shell
U="http://127.0.0.1:3000/api/v1/bookmarks?tag=mdn&sort=title"
curl.exe -s -i "$U&order=desc&limit=2"
```

HTTP

```
HTTP/1.1 200 OK
X-Request-Id: ccf33a86-ee78-4f9b-b40b-47172ec18cfa
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 99
X-RateLimit-Reset: 1791671396
Content-Type: application/json; charset=utf-8
Content-Length: 823
Link: <http://127.0.0.1:3000/api/v1/bookmarks?limit=2&cursor=eyJvZmZzZXQiOjJ9>; rel="next"
Date: Sat, 10 Oct 2026 22:28:55 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "data": [
    {
      "id": "bm_001",
      "url": "https://developer.mozilla.org/en-US/docs/Web/HTTP",
      "title": "MDN HTTP reference",
      "description": "Protocol, methods, headers, status codes",
      "tags": [
        "http",
        "mdn"
      ],
      "favorite": true,
      "createdAt": "2026-09-01T09:00:00.000Z"
    },
    {
      "id": "bm_011",
      "url": "https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API",
      "title": "MDN Fetch API",
      "description": "How browsers make requests from JavaScript",
      "tags": [
        "http",
        "mdn",
        "fetch"
      ],
      "favorite": false,
      "createdAt": "2026-09-15T13:25:00.000Z"
    }
  ],
  "meta": {
    "limit": 2,
    "offset": 0,
    "total": 3,
    "hasMore": true,
    "nextCursor": "eyJvZmZzZXQiOjJ9"
  }
}
```

Three bookmarks carry the tag `mdn`, and they come back in reverse title order. Note that `meta.total` is `3`, the size of the filtered set, not of the whole collection: totals always describe what you are looking at.

Example 2: a free text search.

shell

```
curl.exe -s -i "http://127.0.0.1:3000/api/v1/bookmarks?q=jwt&limit=1"
```

HTTP

```
HTTP/1.1 200 OK
X-Request-Id: 5fd66ef5-9784-480d-8db8-1ee7fb2b6518
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 99
X-RateLimit-Reset: 1791670484
Content-Type: application/json; charset=utf-8
Content-Length: 436
Date: Sat, 10 Oct 2026 22:13:43 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "data": [
    {
      "id": "bm_008",
      "url": "https://jwt.io/introduction",
      "title": "JWT introduction",
      "description": "How access tokens are built and verified",
      "tags": [
        "auth",
        "json"
      ],
      "favorite": false,
      "createdAt": "2026-09-11T09:45:00.000Z"
    }
  ],
  "meta": {
    "limit": 1,
    "offset": 0,
    "total": 1,
    "hasMore": false,
    "nextCursor": null
  }
}
```

The term `jwt` matches the title of `bm_008`. The server searches title, description and url, case insensitively, and reports `hasMore: false` when the filtered set fits in one page.

Example 3: an unknown sort field is a client error.

shell

```
curl.exe -s -i "http://127.0.0.1:3000/api/v1/bookmarks?sort=colour"
```

HTTP

```
HTTP/1.1 400 Bad Request
X-Request-Id: b1c46edd-d539-46bb-85cc-682fc03c5bd0
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 92
X-RateLimit-Reset: 1791669942
Content-Type: application/problem+json; charset=utf-8
Content-Length: 368
Date: Sat, 10 Oct 2026 22:04:51 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "type": "https://learnfree.dev/problems/invalid-query",
  "title": "Invalid query parameter",
  "status": 400,
  "detail": "sort must be one of createdAt, title, url and order one of asc, desc",
  "instance": "/api/v1/bookmarks",
  "errors": [
    {
      "field": "sort",
      "code": "unsupported_value",
      "message": "got \"colour\" / \"asc\""
    }
  ]
}
```

The status line already says `400`; the body says which parameter, what values are legal, and what arrived instead.

Example 4: a page size that is not a number.

shell

```
curl.exe -s -i "http://127.0.0.1:3000/api/v1/bookmarks?limit=abc"
```

HTTP

```
HTTP/1.1 400 Bad Request
X-Request-Id: cf7cfd8d-cfbe-43be-b99d-a26f9a55b905
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 91
X-RateLimit-Reset: 1791669942
Content-Type: application/problem+json; charset=utf-8
Content-Length: 325
Date: Sat, 10 Oct 2026 22:04:51 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "type": "https://learnfree.dev/problems/invalid-query",
  "title": "Invalid query parameter",
  "status": 400,
  "detail": "limit must be an integer between 1 and 100",
  "instance": "/api/v1/bookmarks",
  "errors": [
    {
      "field": "limit",
      "code": "not_a_number",
      "message": "got \"abc\""
    }
  ]
}
```

Example 5: build the query string, do not concatenate it.

JavaScript

```
const base = new URL("http://127.0.0.1:3000/api/v1/bookmarks");
base.searchParams.set("tag", "mdn");
base.searchParams.set("sort", "title");
base.searchParams.set("order", "desc");
base.searchParams.set("limit", "2");
console.log(base.toString());
const shown = [...base.searchParams].map((p) => p.join("="));
console.log(base.pathname + " ->", shown.join("&"));
```

output

```
http://127.0.0.1:3000/api/v1/bookmarks?tag=mdn&sort=title&order=desc&limit=2
/api/v1/bookmarks -> tag=mdn&sort=title&order=desc&limit=2
```

7.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Filters that mutate data	a prefetch changes the database	reads use `GET` only
Ignoring unknown parameters	clients think a filter was applied	reject or document the ignore

Mistake	Why it hurts	Best practice
Sorting without a tie-breaker	unstable pages, duplicated rows	add a unique secondary sort key
`limit=0` meaning "everything"	unbounded responses	default, minimum and maximum, all documented
Filter names that mirror your tables	every schema change is a breaking change	expose a stable vocabulary
Case sensitive search	users get nothing back	normalise both sides
Free text over every column	slow, unpredictable queries	choose the searched fields and say so

- Test the combination of filters together; individually they are easy, together they leak.
- Keep the allowed sort fields in one list in code, and generate the documentation from it.

7.5 Practice exercises

1. Which query parameters would you add to filter bookmarks by a tag and exclude favourites?
2. Why must a default sort order be deterministic?
3. What should the server do with `?limit=abc`, and what should the body contain?
4. Give two reasons a `?q=` search should be case insensitive.
5. A client calls `GET /api/v1/bookmarks?favourite=true` (British spelling) and gets unfiltered results. Why is that bad?

Solutions

1. `?tag=http&favorite=false`.
2. Because pagination depends on it: without a tie-breaker, rows can swap places between two requests, and a cursor then skips or repeats items.
3. Answer `400` with a problem document naming the `limit` parameter, the rule ("integer between 1 and 100") and the value that arrived.
4. Users do not know how the title was typed, and the server should not care about the case of a search term.
5. A misspelled parameter is silently ignored, so the client shows "no favourites" as if it were real data. Either accept both spellings or reject unknown parameters.

7.6 Mini test

1. Which parameter holds a free text search? A) `sort` B) `q` C) `limit` D) `cursor`
2. What does `?order=desc` change? A) the page size B) the direction of the sort C) the filter D) the cursor

3. Which answer fits `?sort=colour`? A) `200` with default sort B) `400` with a problem document C) `404` D) `500`
4. True or false: `meta.total` should count the whole table even when filters are applied.
5. Why does a sort need a tie-breaker?
6. What does `?limit=1000` do when the documented maximum is 100?
7. Why must filters never change server state?

Answer key and explanations

1. B - `q` is the conventional name for a search term.
2. B - `order` only chooses the direction; `sort` chooses the field.
3. B - an unknown field is a client error, and the body names the allowed values.
4. False - it counts the filtered set, so the client knows how many pages of results really exist.
5. So that rows with equal keys keep a fixed relative order between requests, which keeps pagination stable.
6. Either clamp to `100` or reject with `400`; both are acceptable, but it must be documented and consistent.
7. A `GET` must be safe: browsers, prefetchers and caches repeat it, and a read that writes data is a bug waiting for a crawler.

Chapter 8 - Pagination

8.1 Summary

- Pagination splits a large collection into pages the client can actually use.
- Offset pagination counts rows (`?limit=20&offset=40`) and is easy to reason about.
- Cursor pagination remembers a position (`?cursor=...`) and stays correct when the data changes.
- Offset pages drift: a row inserted between two requests shifts everything after it.
- Cursor pages are stable but cannot jump to an arbitrary page.
- The `Link` header is the standard way to advertise the next page.
- A page envelope should carry the rows, the page size, the total, and whether more pages exist.
- `limit` needs a default, a minimum and a maximum.
- Total counts are expensive on large tables; consider reporting only `hasMore`.
- Always combine pagination with a deterministic sort order.

8.2 Concepts

Offset pagination

output

```
?limit=3&offset=0     rows 0,1,2
?limit=3&offset=3     rows 3,4,5
?limit=3&offset=6     rows 6,7,8
...
?limit=3&offset=30    past the end -> empty page
```

Strength	Weakness
trivial to implement	the database still reads and discards the skipped rows
jumping to page 7 is one request	pages drift when rows are inserted or deleted
`total` is natural to report	deep offsets get slow

Cursor pagination

output

```
page 1:  ?limit=3                -> nextCursor "eyJvZmZzZXQiOjN9"
page 2:  ?limit=3&cursor=...9    -> nextCursor "eyJvZmZzZXQiOjZ9"
page 3:  ?limit=3&cursor=...6    -> nextCursor null, done
```

A cursor is an opaque token. In the demo API it is base64url JSON holding the offset of the next row, but a client must treat it as a string and never parse it.

Strength	Weakness
stable while data changes	you cannot jump to an arbitrary page
fast on large collections	the token must be returned in every response
hides the implementation	cursors can expire, so plan for a `400`

The `Link` header

RFC 8288 style relation types are the standard way to expose navigation:

HTTP

```
Link: <http://127.0.0.1:3000/api/v1/bookmarks?limit=2&cursor=eyJvZmZzZXQiOjJ9>; rel="next"
```

A client that understands `Link` never has to guess how to build the next URL. Add `rel="prev"`, `rel="first"` and `rel="last"` where they make sense.

8.3 Examples

Example 1: offset pagination.

shell

```
curl.exe -s -i "http://127.0.0.1:3000/api/v1/bookmarks?limit=3&offset=3"
```

HTTP

```
HTTP/1.1 200 OK
X-Request-Id: 4e037b3d-8bcd-432d-9f01-01991eb24a99
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 94
X-RateLimit-Reset: 1791669942
Content-Type: application/json; charset=utf-8
Content-Length: 1088
Link: <http://127.0.0.1:3000/api/v1/bookmarks?limit=3&cursor=eyJvZmZzZXQiOjZ9>; rel="next"
Date: Sat, 10 Oct 2026 22:04:50 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "data": [
    {
      "id": "bm_004",
      "url": "https://restfulapi.net/",
      "title": "REST API tutorial",
      "description": "Resource naming and status code guidance",
      "tags": [
        "rest",
        "http"
      ],
      "favorite": false,
      "createdAt": "2026-09-05T12:05:00.000Z"
    },
    {
      "id": "bm_005",
      "url": "https://jsonapi.org/",
      "title": "JSON:API",
      "description": "A convention for JSON responses and errors",
      "tags": [
        "json",
        "rest"
      ],
      "favorite": false,
      "createdAt": "2026-09-06T07:30:00.000Z"
    },
    {
      "id": "bm_006",
      "url": "https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS",
      "title": "MDN CORS",
      "description": "Cross-origin requests and the preflight dance",
      "tags": [
        "http",
        "mdn",
        "security"
      ],
      "favorite": true,
      "createdAt": "2026-09-08T16:20:00.000Z"
    }
  ],
```

HTTP (continued)

```
  "meta": {
    "limit": 3,
    "offset": 3,
    "total": 12,
    "hasMore": true,
    "nextCursor": "eyJvZmZzZXQiOjZ9"
  }
}
```

Rows 4 to 6 of 12. `offset` is echoed back so the client can show "showing 4-6 of 12", and `hasMore` tells it whether to ask again.

Example 2: the same collection through a cursor.

shell

```
curl.exe -s -i "http://127.0.0.1:3000/api/v1/bookmarks?limit=2&cursor=eyJvZmZzZXQiOjJ9"
```

HTTP

```
HTTP/1.1 200 OK
X-Request-Id: 65d6d8a2-848a-4bc7-9dc8-be0ce2840d68
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 93
X-RateLimit-Reset: 1791669942
Content-Type: application/json; charset=utf-8
Content-Length: 762
Link: <http://127.0.0.1:3000/api/v1/bookmarks?limit=2&cursor=eyJvZmZzZXQiOjR9>; rel="next"
Date: Sat, 10 Oct 2026 22:04:51 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "data": [
    {
      "id": "bm_003",
      "url": "https://www.rfc-editor.org/rfc/rfc9110.html",
      "title": "RFC 9110",
      "description": "HTTP Semantics, the normative method rules",
      "tags": [
        "http",
        "rfc"
      ],
      "favorite": false,
      "createdAt": "2026-09-03T08:40:00.000Z"
    },
    {
      "id": "bm_004",
      "url": "https://restfulapi.net/",
      "title": "REST API tutorial",
      "description": "Resource naming and status code guidance",
      "tags": [
        "rest",
        "http"
      ],
      "favorite": false,
      "createdAt": "2026-09-05T12:05:00.000Z"
    }
  ],
  "meta": {
    "limit": 2,
    "offset": 2,
    "total": 12,
    "hasMore": true,
    "nextCursor": "eyJvZmZzZXQiOjR9"
  }
}
```

The cursor `eyJvZmZzZXQiOjJ9` decodes to `{"offset":2}`, which is why the page starts at the third row. The response hands back the cursor for the following page, both in `meta.nextCursor` and in the `Link` header, so a client that understands either one can continue.

Example 3: a cursor the server cannot read.

shell

```
curl.exe -s -i "http://127.0.0.1:3000/api/v1/bookmarks?cursor=nope"
```

HTTP

```
HTTP/1.1 400 Bad Request
X-Request-Id: 6472a315-4208-4ddd-8797-68e9696b4a8b
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 90
X-RateLimit-Reset: 1791669942
Content-Type: application/problem+json; charset=utf-8
Content-Length: 317
Date: Sat, 10 Oct 2026 22:04:51 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "type": "https://learnfree.dev/problems/invalid-query",
  "title": "Invalid cursor",
  "status": 400,
  "detail": "the cursor could not be decoded",
  "instance": "/api/v1/bookmarks",
  "errors": [
    {
      "field": "cursor",
      "code": "undecodable",
      "message": "expected base64url JSON"
    }
  ]
}
```

A cursor is a token, not a number. If a client hand-edits one, the honest answer is `400`, not a silently empty page.

Example 4: walking a whole collection with a loop.

JavaScript

```
async function walkAll(url) {
  const seen = [];
  let next = url;
  while (next) {
    const response = await fetch(next, { headers: { Accept: "application/json" } });
    const page = await response.json();
    seen.push(...page.data.map((b) => b.id));
    next = page.meta.hasMore
      ? `http://127.0.0.1:3000/api/v1/bookmarks?limit=4&cursor=${page.meta.nextCursor}`
      : null;
  }
  return seen;
}

const ids = await walkAll("http://127.0.0.1:3000/api/v1/bookmarks?limit=4");
console.log("ids          :", ids.join(", "));
console.log("count        :", ids.length);
console.log("unique       :", new Set(ids).size);
```

output

```
first       : bm_001
last        : bm_012
count       : 12
unique      : 12
```

The loop only follows `meta.nextCursor`, so it works whether the API returns links, offsets or tokens. Comparing `count` with `unique` is the cheapest regression test for a broken cursor.Example 5: defaults, caps and empty pages.

JavaScript

```javascript
const params = new URLSearchParams("limit=&offset=&cursor=");
const limit = Number(params.get("limit") || 20);
const offset = Number(params.get("offset") || 0);
console.log("default limit:", limit);
console.log("default offset:", offset);
console.log("clamped      :", Math.min(Math.max(limit, 1), 100));
console.log("page past end:", Math.min(offset, 12), "of 12");
```

output

```
default limit: 20
default offset: 0
clamped      : 20
page past end: 12 of 12
```

An empty `data` array with `hasMore: false` is the correct answer for a page beyond the end; `null` or an error is not.

8.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
No default `limit`	clients get a million rows	default, minimum and maximum
Offset pagination on a huge table	the query reads everything it skips	cursor pagination for large collections
Pagination without a sort order	rows jump between pages	always sort, always with a tie-breaker
Total counts on every request	expensive aggregations	report `hasMore`, or cache the total
Exposing the cursor's internals	clients start parsing it	treat it as an opaque token
Changing the page size mid-walk	cursors become meaningless	keep `limit` constant during a walk
Returning `null` for an empty page	clients crash on `map`	`[]` with `hasMore: false`

- Document the paging parameters in the OpenAPI file, exactly as the demo API does in chapter 10.
- When you switch from offset to cursor, support both for one version and remove offset in the next.

8.5 Practice exercises

1. Give one advantage and one disadvantage of offset pagination.
2. Why is a cursor more stable than an offset when rows are being inserted?
3. What are the four things a page envelope should tell the client?
4. What should the server return for `?limit=5&offset=1000` on a 12 row collection?
5. Why must `limit` stay constant while a client walks a cursor?

Solutions

1. Advantage: you can jump straight to any page. Disadvantage: deep offsets make the database read and discard everything before the page.
2. The cursor points at a position in a deterministic order, so a new row inserted before it does not move the reader's place; an offset is a count, and every insertion shifts what it counts to.
3. The page size, the rows themselves, how many rows exist in total, and whether more pages follow, plus the token for the next page.
4. `200` with `"data": []` and `"hasMore": false`; an empty page is a valid answer, not an error.
5. Because the cursor encodes a position in the order, and changing the page size mid-walk makes the position meaningless.

8.6 Mini test

1. Which parameter holds the page size? A) `offset` B) `limit` C) `cursor` D) `sort`
2. Which pagination style can jump to page 42 directly? A) cursor B) offset C) both D) neither
3. Which header advertises the URL of the next page? A) `Location` B) `Link` C) `Content-Range` D) `Next-Page`
4. True or false: a cursor is meant to be decoded by the client.
5. What does `hasMore: true` tell the client?
6. Why does pagination require a deterministic sort order?
7. A collection returns 10 000 rows and the client asks for all of them. What should the API do?

Answer key and explanations

1. B - `limit` is the page size; `offset` is where the page starts.
2. B - offset pagination counts rows, so any page number is one request away.
3. B - `Link` with `rel="next"` is the standard way to publish the next URL.
4. False - the cursor is an opaque token; decoding it couples the client to server internals.
5. That at least one more page exists, so the client should ask again with the returned cursor.
6. Without a fixed order, the same row can appear on two pages or on none.
7. Cap the response with a default and a maximum `limit`, and make the client walk pages instead of receiving everything at once.

Chapter 9 - Authentication and authorization

9.1 Summary

- Authentication answers "who is calling"; authorization answers "what may they do".
- API keys are long lived secrets, easy to use and easy to leak.
- A JWT is three base64url parts: a header, a payload of claims and a signature.
- The signature is what makes a JWT trustworthy; the payload is readable by anyone.
- Access tokens expire, which is why they carry `exp` and why clients must handle a `401`.
- OAuth 2.0 delegates access without sharing a password, using short lived codes and tokens.
- The authorization code flow with PKCE is the right default for public clients such as browsers and mobile apps.
- Client credentials is the right default for server to server calls.
- Credentials belong in headers, never in URLs, because URLs land in logs and history.
- Everything in this chapter assumes HTTPS; without it, every token on the wire is public.

9.2 Concepts

Keys, tokens and sessions

Mechanism	Shape	Good for	Watch out for
API key	one long string in a header	server to server, quick starts	no expiry, no scopes, easy to leak
Basic auth	`user:password` base64	internal tools only	needs HTTPS, no expiry
Bearer token (JWT)	signed JSON in a header	user sessions, micro services	revocation is hard
OAuth 2.0 code	short lived code exchanged for a token	third party access	more moving parts
Session cookie	opaque id in a cookie	browser apps on your own domain	CSRF protection needed

How a JWT is built

output

```
header        payload                     signature
+--------+    +-----------------------+   +------------------+
| alg    | -> | claims (iss, sub, exp)  | ->| HMAC-SHA256 over  |
| typ    |    | scope, custom fields    |   | header + payload  |
+--------+    +-----------------------+   +------------------+
    |                 |                           |
base64url        base64url                   base64url
    |                 |                           |
    +-----------+-----+---------------------------+
                |
          header.payload.signature
```

The two first parts are not encrypted, only encoded. Anyone who sees the token can read the claims; the signature only proves nobody changed them.

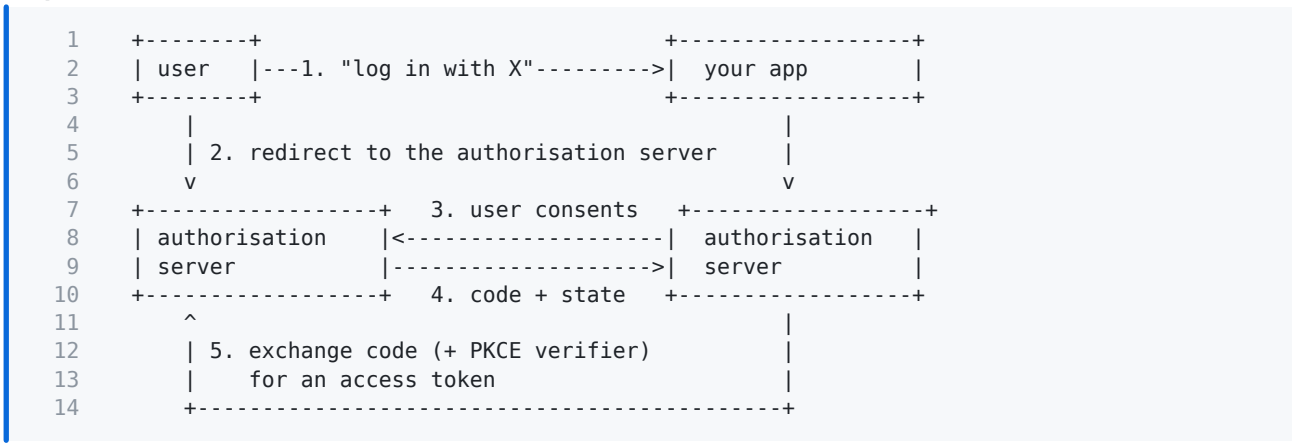
The claims worth knowing

Claim	Meaning
`iss`	who issued the token
`sub`	who the token is about
`aud`	which service should accept it
`exp`	expiry, seconds since the epoch
`iat`	when it was issued
`nbf`	not valid before
`scope`	what the caller may do

Verification is a checklist: split into three parts, recompute the signature, reject if it differs, then check `exp`, `aud` and `iss`. A token that fails any step gets a `401` and a `WWW-Authenticate` challenge.

OAuth 2.0 in one picture

output



Flow	Who it is for	Key idea
Authorization code + PKCE	browsers, mobile, desktop	a short lived code, never the password, and a proof that the caller is the same one that started
Client credentials	backend services	the client authenticates as itself, there is no user
Device code	TVs and CLIs	the user approves on a second device
Implicit	nobody, any more	tokens in URLs, deprecated

PKCE adds a `code_challenge` to the authorisation request and a `code_verifier` to the token request, so a stolen code is useless on its own.

9.3 Examples

Example 1: what a JWT looks like on the inside.

shell

```shell
node jwt-parts.mjs
```

output

```
a JWT has 155 characters, 3 parts and 2 dots

1. header  (algorithm and type)
   encoded : eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
   decoded : {"alg":"HS256","typ":"JWT"}

2. payload (the claims)
   encoded : eyJpYXQiOjE3OTE2Njk5NTEsImV4cCI6MTc5MTY3MzU1MS
   encoded : wic3ViIjoiZGVtby1jbGllbnQifQ
   decoded : {"iat":1791669951,"exp":1791673551,"sub":"demo-client"}

3. signature (HMAC-SHA256 over "<header>.<payload>")
   encoded : o9Nn3Zo4IXPuK8EfhxdEuK8T2kckbqdJyvmWmLDKa-c

the token is sent as: Authorization: Bearer <token>
```

The long encoded values are wrapped here to fit the page. The header says which algorithm was used, the payload says who the token is about and when it dies, and the signature binds the two together.

Example 2: encoding and decoding a part by hand.

JavaScript

```javascript
const header = '{"alg":"HS256","typ":"JWT"}';
const encoded = Buffer.from(header, "utf8").toString("base64url");
console.log("encoded    :", encoded);
console.log("decoded    :", Buffer.from(encoded, "base64url").toString("utf8"));
console.log("padding    :", Buffer.from("hi", "utf8").toString("base64"));
```

output

```
encoded    : eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
decoded    : {"alg":"HS256","typ":"JWT"}
padding    : aGk=
```

`base64url` is base64 with `-` and `_` instead of `+` and `/` and no padding, so the value is safe in a URL and in a header.

Example 3: exchanging an API key for a token.

shell

```
curl.exe -s -D - -o NUL -X POST http://127.0.0.1:3000/api/v1/tokens \
  -H "X-API-Key: lf-demo-key-123"
```

HTTP

```
HTTP/1.1 200 OK
X-Request-Id: 087ee459-5ba8-46c0-ba36-490dc4cc73b9
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 99
X-RateLimit-Reset: 1791670676
Content-Type: application/json; charset=utf-8
Content-Length: 262
Cache-Control: no-store
Date: Sat, 10 Oct 2026 22:16:55 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

`-D -` prints the response headers and `-o NUL` discards the body, which is how you inspect a credential response without copying a secret into your terminal history. Note `Cache-Control: no-store`: a token must never be cached.

Example 4: using the token, and what happens when it is wrong.

shell

```
curl.exe -s -X POST http://127.0.0.1:3000/api/v1/tokens -o token.json \
  -H "X-API-Key: lf-demo-key-123"
TOKEN=$(node -e "console.log(require('./token.json').access_token)")
AUTH="Authorization: Bearer $TOKEN"
curl.exe -s -i http://127.0.0.1:3000/api/v1/me -H "$AUTH"
```

HTTP

```
HTTP/1.1 200 OK
X-Request-Id: 19e368cc-4e64-4caf-ba5d-696c47a9f42a
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 95
X-RateLimit-Reset: 1791671319
Content-Type: application/json; charset=utf-8
Content-Length: 60
Date: Sat, 10 Oct 2026 22:27:47 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "client": "demo-client",
  "scope": "bookmarks:write"
}
```

Keeping the token in a shell variable keeps it out of the command line of the second request, and out of your shell history. The token response also carries `Cache-Control: no-store`, which forbids any cache from keeping a copy.

The same endpoint with a broken token:

shell

```
curl.exe -s -i http://127.0.0.1:3000/api/v1/me -H "Authorization: Bearer not.a.token"
```

HTTP

```
HTTP/1.1 401 Unauthorized
X-Request-Id: 778aa00f-8d34-4484-89da-4516f1e95b35
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 86
X-RateLimit-Reset: 1791669942
Content-Type: application/problem+json; charset=utf-8
Content-Length: 182
WWW-Authenticate: Bearer realm="bookmarks", error="invalid_token"
Date: Sat, 10 Oct 2026 22:05:14 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "type": "https://learnfree.dev/problems/invalid-token",
  "title": "Invalid access token",
  "status": 401,
  "detail": "the bearer token could not be accepted: bad_signature"
}
```

A token minted with an expiry in the past answers with the same status and `detail: "the bearer token could not be accepted: expired"`, so a client can tell "refresh me" from "you are not who you say you are".

Example 5: the shape of an OAuth 2.0 authorisation request.

output

```
GET https://authorization.example.com/authorize
    ?response_type=code
    &client_id=bookmarks-demo
    &redirect_uri=https://app.example.com/callback
    &scope=bookmarks:write
    &state=8f1c2b7e1d4a4a1f9a2e2c4b8f0d1111
    &code_challenge=E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM
    &code_challenge_method=S256
```

output

```
the browser is redirected to:

https://app.example.com/callback?code=4c1f...&state=8f1c2b7e1d4a4a1f9a2e2c4b8f0d1111

the app then POSTs to the token endpoint:

POST /oauth/token
    grant_type=authorization_code
    &code=4c1f...
    &redirect_uri=https://app.example.com/callback
    &client_id=bookmarks-demo
    &code_verifier=the-random-string-that-produced-the-challenge
```

`state` comes back unchanged, which proves the response belongs to the request the app started. The `code_verifier` is never sent in the browser step; keeping it on the server side is what PKCE protects. No password ever reaches the app, and the code is valid once, for a short time.

9.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Tokens in query strings	they end up in logs, history and referrers	use the `Authorization` header
Long lived access tokens	a leak stays useful for months	minutes to hours, then refresh
Trusting a JWT payload without checking the signature	anyone can forge claims	verify signature, `exp`, `aud` and `iss`
One scope for everything	a read client can delete data	issue the narrowest scope
Storing tokens in `localStorage`	any script on the page can read them	httpOnly cookies, or secure platform storage
Reusing an OAuth code	replay attacks	single use, short lived, bound to PKCE
Accepting `alg: none`	the token is unsigned and trivially faked	pin the algorithm you expect
Returning different `401` bodies	clients cannot tell retry from give up	one problem document, one challenge header

- Rotate keys and secrets on a schedule, and make rotation a documented procedure.

- Log authentication failures, never the credentials themselves.

9.5 Practice exercises

1. What is the difference between authentication and authorization?
2. Name the three parts of a JWT and say which one is signed.
3. Why is a JWT payload said to be readable by anyone?
4. What does PKCE protect against, and where does the verifier live?
5. Which two headers must never carry a credential, and why?

Solutions

1. Authentication establishes identity; authorization decides what that identity may do.
2. Header, payload and signature. The signature covers the first two parts and proves they were not modified.
3. Because the header and payload are base64url encoded, not encrypted: decoding them needs no key.
4. It protects the authorisation code from being replayed by whoever intercepted it; the verifier stays with the client that started the flow and only the challenge travels in the browser.
5. The URL and, in particular, the query string: URLs are logged by proxies, kept in browser history and sent as referrers.

9.6 Mini test

1. Which HTTP header carries a bearer token? A) `X-API-Key` B) `Authorization` C) `Cookie` D) `X-Auth-Token`
2. Which part of a JWT is protected against modification? A) the header B) the payload C) the signature D) all three
3. Which claim states when a token stops being valid? A) `iat` B) `sub` C) `exp` D) `iss`
4. True or false: a JWT payload is encrypted, so only the server can read it.
5. Which OAuth 2.0 flow suits a single page application best? A) implicit B) client credentials C) authorization code with PKCE D) resource owner password
6. What is the purpose of the `state` parameter in an OAuth redirect?
7. Why must credentials never appear in a URL?

Answer key and explanations

1. B - `Authorization: Bearer <token>` is the standard form.
2. C - the signature covers header and payload, so tampering invalidates it.
3. C - `exp` is the expiry timestamp.
4. False - the parts are encoded, not encrypted; the signature only proves integrity.

5. C - PKCE binds the code exchange to the client that started it, which is exactly the browser's problem.
6. It binds the callback to the request the app started, blocking cross site request forgery on the redirect.
7. URLs are recorded in access logs, browser history and `Referer` headers, so a credential in a URL is a credential you have published.

Chapter 10 - Documenting with OpenAPI

10.1 Summary

- OpenAPI is a machine readable description of an HTTP API, written as YAML or JSON.
- Version 3.1 describes the same schemas as JSON Schema, which makes validation reusable.
- The document has a fixed skeleton: `openapi`, `info`, `servers`, `paths` and `components`.
- Each path holds one or more operations, and each operation has parameters, a request body and responses.
- Schemas live in `components`, so every response can reference the same definition.
- Responses are documented per status code, including the error ones.
- Security schemes are declared once and applied to the operations that need them.
- Examples inside the document become the source for SDKs, mock servers and docs pages.
- A lint pass over the document catches dangling references and missing descriptions.
- The document is a build artefact: review it in code review like any other file.

10.2 Concepts

The skeleton

output

```
openapi: 3.1.0
info            title, version, description, contact
servers         the base URLs the document describes
security        the default credential scheme
tags            grouping for the documentation page
paths           one entry per URL, one operation per method
components      reusable schemas, parameters, responses, security schemes
```

What an operation declares

Field	Purpose
`tags`	group the operation on the docs page
`summary`	one line, shown in the operation list

Field	Purpose
`description`	the detail, in Markdown
`operationId`	a unique name used to generate client methods
`parameters`	path, query, header and cookie inputs
`requestBody`	the payload schema and its media types
`responses`	one entry per status code, each with a schema
`security`	which schemes this operation needs
`deprecated`	marks an operation as on its way out

`$ref`, the reason OpenAPI stays small

Every schema and reusable response is defined once under `components` and referenced everywhere else:

YAML

```yaml
"404":
  $ref: "#/components/responses/NotFound"
```

Rename the definition in one place and every operation follows. Tools that generate clients and servers follow the same references, so the generated code matches the document exactly.

10.3 Examples

Example 1: the smallest useful document.

YAML

```yaml
openapi: 3.1.0
info:
  title: Bookmarks API
  version: 1.0.0
servers:
  - url: http://127.0.0.1:3000/api/v1
paths:
  /health:
    get:
      operationId: getHealth
      responses:
        "200":
          description: The service is running
          content:
            application/json:
              schema:
                type: object
                required: [status, version]
                properties:
                  status:
                    type: string
                    examples: [ok]
                  version:
                    type: string
                    examples: ["1.0.0"]
```

Four keys and one path are enough to generate a client stub. Everything else is refinement.

Example 2: the real head of the demo document.

YAML

```yaml
openapi: 3.1.0
info:
  title: Bookmarks API
  version: 1.0.0
  summary: A small bookmark store used to teach API design
  description: |
    This document describes the demo API built in Vol. 15. It uses a
    resource oriented URL layout, JSON request and response bodies, an
    RFC 9457 problem document for every error and three ways to page
    through a collection.
  contact:
    name: Learn for Free
    url: https://developer.mozilla.org/en-US/docs/Web/HTTP
servers:
  - url: http://127.0.0.1:3000/api/v1
    description: local demo server
security:
  - apiKey: []
tags:
  - name: bookmarks
    description: Create, read, update and delete bookmarks
  - name: auth
    description: Exchange an API key for a bearer token
```

`security: - apiKey: []` at the top means "every operation needs a key unless it says otherwise", which is how `/health` ends up public by setting `security: []` on itself.

Example 3: query parameters and the page schema.

YAML

```yaml
      parameters:
        - name: sort
          in: query
          description: Field used to order the result
          schema:
            type: string
            enum: [createdAt, title, url]
            default: createdAt
        - name: order
          in: query
          description: Sort direction
          schema:
            type: string
            enum: [asc, desc]
            default: asc
        - name: limit
          in: query
          description: Page size
          schema:
            type: integer
            minimum: 1
            maximum: 100
            default: 20
        - name: cursor
          in: query
          description: Opaque cursor returned by the previous page (cursor pagination)
          schema:
            type: string
          responses:
            "200":
              description: A page of bookmarks
              headers:
                Link:
                  description: Relation link to the next page, when more exist
                  schema:
                    type: string
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/BookmarkPage"
        "400":
          $ref: "#/components/responses/BadQuery"
        "429":
          $ref: "#/components/responses/RateLimited"
```

The `enum` on `sort` and `order` is the same list the server validates against, so the document and the code cannot drift apart silently.

Example 4: the schemas behind the responses.

YAML

```
    Bookmark:
      type: object
      required: [id, url, title, tags, favorite, createdAt]
      properties:
        id:
          type: string
          examples: [bm_003]
        url:
          type: string
          format: uri
          examples: ["https://www.rfc-editor.org/rfc/rfc9110.html"]
        title:
          type: string
          maxLength: 200
          examples: ["RFC 9110"]
        description:
          type: string
          default: ""
          examples: ["HTTP Semantics, the normative method rules"]
        tags:
          type: array
          maxItems: 10
          items:
            type: string
          examples: [[http, rfc]]
        favorite:
          type: boolean
        createdAt:
          type: string
          format: date-time
          examples: ["2026-09-03T08:40:00.000Z"]
```

The page envelope and the problem document are defined the same way:

YAML

```
    BookmarkPage:
      type: object
      required: [data, meta]
      properties:
        data:
          type: array
          items:
            $ref: "#/components/schemas/Bookmark"
        meta:
          type: object
          required: [limit, offset, total, hasMore]
          properties:
            limit:
              type: integer
            offset:
              type: integer
            total:
              type: integer
            hasMore:
              type: boolean
            nextCursor:
              type: [string, "null"]
    Problem:
      type: object
      description: RFC 9457 problem details
      required: [type, title, status]
```

`type: [string, "null"]` is JSON Schema 2020-12 syntax for "nullable", new in OpenAPI 3.1; the older `nullable: true` keyword belongs to 3.0 documents.

Example 5: security schemes, and linting the document.

YAML

```yaml
  securitySchemes:
    apiKey:
      type: apiKey
      in: header
      name: X-API-Key
      description: Project key issued by the API owner
    bearer:
      type: http
      scheme: bearer
      bearerFormat: JWT
```

The document is only useful if it parses and holds together. The first thing the lint script does is parse it with `yaml.safe_load`, so a syntax error fails before any check runs:

shell

```
python lint_openapi.py openapi.yaml
```

output

```
checked openapi.yaml
paths      : 5
operations : 8
schemas    : 5
references : 28
PROBLEMS   : 0
```

The lint script walks every `$ref`, checks that each path starts with `/`, that every operation has responses, and that every name in `required` exists in `properties`. It is fifty lines and it catches the mistakes that make generated clients crash.

10.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Writing the document after the code	it describes a version that no longer exists	write it beside the code, in the same review
Duplicating schemas instead of `$ref`	the drift starts immediately	one definition, many references
Documenting only `200`	clients do not know what errors to expect	document every status you can return
No `operationId`	generated clients get ugly method names	unique, stable, camelCase ids
Examples that contradict the schema	generated SDKs send invalid data	validate the document in CI

Mistake	Why it hurts	Best practice
Describing an endpoint you have not built	the doc becomes fiction	delete or mark `deprecated`
Putting credentials in `example` values	the document leaks secrets	placeholders only

- Keep the document in the repository next to the routes it describes.
- Add the lint command to your test runner; a broken document should fail the build.

10.5 Practice exercises

1. Which top level key holds the base URLs of an API?
2. What does a `$ref` pointing at `#/components/schemas/Bookmark` mean?
3. Why must error responses be documented too?
4. What is an `operationId` used for?
5. How do you mark a field as nullable in OpenAPI 3.1?

Solutions

1. `servers`.
2. That the schema is defined once under `components.schemas` and referenced from here, so a change to the definition applies everywhere it is used.
3. Because clients must handle failures deliberately: the documented status codes are what a generated client and a human reader both rely on.
4. It is the stable, unique name a code generator uses for the method that calls this operation.
5. With a type array, `type: [string, "null"]`, which is JSON Schema 2020-12 syntax.

10.6 Mini test

1. Which version of OpenAPI uses JSON Schema 2020-12 for its schemas? A) 2.0 B) 3.0 C) 3.1 D) 3.2
2. Which field gives an operation a stable name for code generation? A) `summary` B) `operationId` C) `tags` D) `description`
3. Which top level key holds reusable schemas? A) `paths` B) `components` C) `servers` D) `definitions`
4. True or false: a single YAML document can describe several base URLs.
5. What does the document's top level `security` key do?
6. Which keyword replaces `nullable: true` from OpenAPI 3.0?
7. Why should an OpenAPI document live in the same repository as the code?

Answer key and explanations

1. C - 3.1 moved schema handling over to JSON Schema 2020-12.
2. B - `operationId` is the generated method name.
3. B - `components` holds schemas, parameters, responses and security schemes.
4. True - `servers` is a list, for example a production and a staging URL.
5. It sets the default security requirement for every operation, which each operation can override.
6. A type array such as `type: [string, "null"]`.
7. So the document changes in the same commit as the routes, and so CI can lint and validate it.

Chapter 11 - Versioning, rate limiting and cross-origin security

11.1 Summary

- Versioning is a promise: old clients keep working while new clients use new shapes.
- A breaking change needs a new version; adding a field or an endpoint does not.
- URI versioning (`/api/v1/...`) is the most visible and the easiest to route.
- Header versioning (`Accept: application/vnd.example.v2+json`) keeps URLs clean but hides the version.
- Rate limiting protects a service by refusing excess work, and it must be visible to the client.
- A quota response carries `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset` and, on a `429`, `Retry-After`.
- Clients must slow down when the remaining budget is low instead of waiting for a `429`.
- CORS is the browser's permission system for cross-origin calls, and it runs a preflight before non-simple requests.
- A preflight is answered with `Access-Control-Allow-Methods`, `Access-Control-Allow-Headers` and `Access-Control-Max-Age`.
- Security headers such as `X-Content-Type-Options` and `Referrer-Policy` cost nothing and close real holes.

11.2 Concepts

What counts as a breaking change

Change	Breaking?
Removing a field from a response	yes
Renaming a field	yes

Change	Breaking?
Changing a field's type	yes
Making an optional field required in a request	yes
Tightening validation	yes
Adding a response field	no
Adding an optional request field	no
Adding an endpoint	no
Adding a new enum value that clients must tolerate	usually no

If a change is breaking, either add it beside the old thing or publish a new version. Deprecation is a process, not an event: announce, add a `Deprecation` header, monitor usage, remove.

Where the version lives

Strategy	Example	Strength	Weakness
URI path	`/api/v1/bookmarks`	obvious, cacheable, easy to route	version in every URL
Query parameter	`?version=2`	easy to add	ignored by caches, easy to forget
Header	`Accept: application/vnd.learnfree.v2+json`	clean URLs	invisible in logs, harder to test by hand
Host name	`v2.api.example.com`	separate deploy	DNS and certificates to manage

URI versioning wins for public APIs because a human can read the URL and say what will happen.

Rate limiting

output

```
client                        server
  |  GET /bookmarks              |
  |----------------------------->|  bucket: 96 of 100 used
  |  200 + X-RateLimit-*         |
  |<-----------------------------|
  |  GET /bookmarks              |
  |----------------------------->|  bucket: 100 of 100 used
  |  429 + Retry-After: 60       |
  |<-----------------------------|
  |  ... waits 60 seconds ...    |
  |  GET /bookmarks              |
  |----------------------------->|  new window, bucket reset
```

Limits can be per API key, per IP, per endpoint or per user; a public API usually combines them. Whatever the bucket, the client needs three numbers: how much is allowed, how much

is left, and when the window resets.

CORS in one exchange

output

```
browser on http://localhost:5173 wants to POST JSON to :3000

1. OPTIONS /api/v1/bookmarks
   Origin: http://localhost:5173
   Access-Control-Request-Method: POST
   Access-Control-Request-Headers: content-type,x-api-key

2. 204 No Content
   Access-Control-Allow-Origin: http://localhost:5173
   Access-Control-Allow-Methods: GET, POST, PATCH, DELETE, OPTIONS
   Access-Control-Allow-Headers: Content-Type, X-API-Key, Authorization
   Access-Control-Max-Age: 600

3. the real POST goes ahead, with the Allow-Origin header echoed
```

A "simple" request (`GET` with plain headers) skips the preflight. JSON bodies and custom headers such as `X-API-Key` do not, which is exactly why the demo API answers `OPTIONS`. `Access-Control-Max-Age` caches the permission so the preflight is not repeated on every call.

11.3 Examples

Example 1: the same API in two versions.

shell

```
curl.exe -s -i "http://127.0.0.1:3000/api/v2/bookmarks?limit=1"
```

HTTP

```
HTTP/1.1 200 OK
X-Request-Id: eac2f6aa-dc7b-416c-84aa-69a3eada94f4
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 99
X-RateLimit-Reset: 1791670003
Content-Type: application/json; charset=utf-8
Content-Length: 477
Date: Sat, 10 Oct 2026 22:05:42 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "items": [
    {
      "id": "bm_001",
      "url": "https://developer.mozilla.org/en-US/docs/Web/HTTP",
      "title": "MDN HTTP reference",
      "description": "Protocol, methods, headers, status codes",
      "tags": [
        "http",
        "mdn"
      ],
      "favorite": true,
      "createdAt": "2026-09-01T09:00:00.000Z"
    }
  ],
  "pageInfo": {
    "limit": 1,
    "offset": 0,
    "total": 12,
    "hasMore": true,
    "nextCursor": "eyJvZmZzZXQiOjF9"
  }
}
```

Version 2 renames the envelope keys from `data`/`meta` to `items`/`pageInfo`. Version 1 still answers exactly as before, which is the whole point: a client written last year does not break today.

Example 2: the same change delivered through a header.

shell

```
curl.exe -s -i "http://127.0.0.1:3000/api/v1/bookmarks?limit=1" \
  -H "Accept: application/vnd.learnfree.v2+json"
```

HTTP

```
HTTP/1.1 200 OK
X-Request-Id: ec0c5a27-dddf-4c6c-b337-5e4db61788f1
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 98
X-RateLimit-Reset: 1791670003
Content-Type: application/json; charset=utf-8
Content-Length: 477
Date: Sat, 10 Oct 2026 22:05:43 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "items": [
    {
      "id": "bm_001",
      "url": "https://developer.mozilla.org/en-US/docs/Web/HTTP",
      "title": "MDN HTTP reference",
      "description": "Protocol, methods, headers, status codes",
      "tags": [
        "http",
        "mdn"
      ],
      "favorite": true,
      "createdAt": "2026-09-01T09:00:00.000Z"
    }
  ],
  "pageInfo": {
    "limit": 1,
    "offset": 0,
    "total": 12,
    "hasMore": true,
    "nextCursor": "eyJvZmZzZXQiOjF9"
  }
}
```

The URL is the version 1 one, the caller still gets the version 2 envelope, and `Vary: Origin` shows how a cache is told that the answer depends on the request.

Example 3: what a quota looks like before it runs out.

shell

```
curl.exe -s -D - -o NUL http://127.0.0.1:3000/api/v1/bookmarks?limit=1
```

HTTP

```
HTTP/1.1 200 OK
X-Request-Id: 799e7c85-7151-4a96-91e6-aa131c669431
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 99
X-RateLimit-Reset: 1791669942
Content-Type: application/json; charset=utf-8
Content-Length: 762
Date: Sat, 10 Oct 2026 22:04:41 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Three headers, three facts: the window allows 100 requests, 99 are left, and the window resets at unix time `1791669942`. A client that logs these can slow down before it is refused.

Example 4: what a `429` looks like when the quota is gone.

shell

```
node flood.mjs 105
```

output

```
sent 105 GET /api/v1/health requests with X-API-Key: lf-flood-key-456
  200 -> 100 response(s)
  429 -> 5 response(s)
first 429 after request number 101
```

shell

```
curl.exe -s -i http://127.0.0.1:3000/api/v1/health -H "X-API-Key: lf-flood-key-456"
```

HTTP

```
HTTP/1.1 429 Too Many Requests
X-Request-Id: ad4ccf0e-0ab5-4f2a-969a-be590a08a5e2
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
Content-Type: application/problem+json; charset=utf-8
Content-Length: 210
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1791670004
Retry-After: 60
Date: Sat, 10 Oct 2026 22:05:43 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "type": "https://learnfree.dev/problems/rate-limited",
  "title": "Too many requests",
  "status": 429,
  "detail": "The limit of 100 requests per minute has been reached",
  "instance": "/api/v1/health"
}
```

`Retry-After: 60` is measured in seconds, and the reset timestamp is there too. A well behaved client waits; a badly behaved one retries immediately and is usually banned.

Example 5: the preflight, and the code that produces it.

shell

```shell
curl.exe -s -i -X OPTIONS "http://127.0.0.1:3000/api/v1/bookmarks" \
  -H "Origin: http://localhost:5173" \
  -H "Access-Control-Request-Method: POST" \
  -H "Access-Control-Request-Headers: content-type,x-api-key"
```

HTTP

```http
HTTP/1.1 204 No Content
X-Request-Id: c41a7c10-99c0-4995-93f9-d0e0750a2304
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
Access-Control-Allow-Methods: GET, POST, PATCH, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, X-API-Key, Authorization
Access-Control-Max-Age: 600
Content-Length: 0
Date: Sat, 10 Oct 2026 22:05:14 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

The handler behind it is short enough to quote in full:

JavaScript

```javascript
if (req.method === "OPTIONS") {
  res.writeHead(204, {
    "Access-Control-Allow-Methods": "GET, POST, PATCH, DELETE, OPTIONS",
    "Access-Control-Allow-Headers": "Content-Type, X-API-Key, Authorization",
    "Access-Control-Max-Age": "600",
    "Content-Length": "0",
  });
  res.end();
  return;
}
```

output

```
(no output; the browser receives 204 and then sends the real POST)
```

Note what is missing: `Access-Control-Allow-Credentials`. Sending cookies or `Authorization` headers cross origin requires it, and it must never be combined with a wildcard origin.

11.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Renaming a field without a new version	every client breaks at once	add the new field, drop the old one later
Versioning in the query string	caches ignore it, clients forget it	version in the path

Mistake	Why it hurts	Best practice
No rate limit headers	clients must guess and get banned	publish limit, remaining and reset
`Retry-After` in milliseconds	clients interpret it as seconds and hammer	always seconds, always a number
`Access-Control-Allow-Origin: *` with credentials	browsers reject the response	echo an explicit allowed origin
Forgetting `OPTIONS` handling	every browser call fails before it starts	answer preflight on every route
Unlimited `limit`	one request can exhaust the service	default, minimum and maximum
Rate limiting by IP only	shared NAT addresses punish everyone	combine key, IP and endpoint buckets

- Treat the version number as part of your public contract and say when support ends.
- Monitor `X-RateLimit-Remaining` in your own client, and log it when you are throttled.

11.5 Practice exercises

1. Which of these is a breaking change: adding a response field, or removing one?
2. Give one advantage and one disadvantage of URI versioning.
3. Which three headers tell a client about its quota before it is throttled?
4. What does `Access-Control-Max-Age: 600` do?
5. Why must a preflight answer list the methods and headers explicitly?

Solutions

1. Removing one. Adding a field is backwards compatible; removing it breaks every client that reads it.
2. Advantage: it is visible, cacheable and easy to route. Disadvantage: the version is part of every URL and cannot be changed quietly.
3. `X-RateLimit-Limit`, `X-RateLimit-Remaining` and `X-RateLimit-Reset`.
4. It tells the browser it may cache the preflight permission for 600 seconds, so the preflight is not repeated on every request.
5. Because the browser refuses to send a request that is not explicitly permitted: the preflight is where the server grants the method and the headers.

11.6 Mini test

1. Which header carries the version in the demo API's second strategy? A) `X-API-Version` B) `Accept` C) `Vary` D) `Location`
2. Which status code means the quota is exhausted? A) `403` B) `409` C) `429` D) `503`

3. Which header says how long to wait before retrying a `429`? A) `Retry-After` B) `X-RateLimit-Reset` C) `Cache-Control` D) `Keep-Alive`
4. True or false: adding a new optional field to a response is a breaking change.
5. Which method performs the CORS preflight?
6. Why is `Access-Control-Allow-Origin: *` unacceptable together with `Access-Control-Allow-Credentials: true`?
7. What does `Vary: Origin` protect against?

Answer key and explanations

1. B - an `Accept` media type such as `application/vnd.learnfree.v2+json` selects the representation.
2. C - `429 Too Many Requests` is the quota answer.
3. A - `Retry-After` is expressed in seconds.
4. False - old clients simply ignore a field they do not know.
5. `OPTIONS`.
6. Because a wildcard would let any site send credentialed requests; browsers reject that combination outright.
7. A shared cache reusing a response for a different `Origin`, which would leak one site's answer to another.

Chapter 12 - Testing APIs

12.1 Summary

- A test asserts one thing: for a given request, this is the status, these are the headers, this is the body.
- Manual testing with curl finds what a user would find; automated tests find what you forgot.
- Check the status code first, then the headers that carry policy, then the body.
- Error paths deserve as many tests as the happy path, because that is where clients break.
- Start the server in the test, on an ephemeral port, so tests never share state with your dev server.
- `node:test` and `node:assert` are in the standard library, so a suite needs no dependencies.
- Assert on the problem document shape: status, `type`, and the `errors` array.
- Keep tests fast and independent: a suite that takes minutes stops being run.
- A failing test must print the expected value and the actual one, which is what good assertions do.
- Run the suite in CI; a test that only runs on your machine is a note, not a test.

12.2 Concepts

The test pyramid for an API

Level	Speed	What it covers
Smoke checks	seconds	is every route still answering at all
Contract tests	minutes	one assertion per status code and per schema
Integration tests	minutes	create, read, update, delete in order, against a real store
Unit tests	milliseconds	validation helpers, cursor encoding, sorting

Most bugs in an API live at the boundaries: a status code that changed, a field that vanished, a header that stopped being sent. Those are cheap to test.

A good assertion

output

```
assert.equal(response.status, 201);               obvious
assert.match(headers.get("location"), /bm_013$/);  precise
assert.deepEqual(page.errors.map(e => e.field),
                 ["title", "url"]);                complete
```

`assert.equal` on the status code is the single most valuable line in an API test. Everything else is detail.

Isolating the server

A test suite must not depend on a server you started by hand. Two options work well:

output

```
in process   import createServer(), listen on port 0, close in after()
out of process spawn node server.mjs <port>, poll until it answers, kill it
```

Port 0 asks the operating system for a free port, so parallel test files never collide. The demo suite uses the first style for everything except the rate limit test, where a fresh process with its own environment is the honest way to test a quota.

12.3 Examples

Example 1: a smoke check you can run in five seconds.

shell

```
node smoke.mjs
```

output

```
ok   health                expected 200, got 200
ok   collection            expected 200, got 200
ok   member                expected 200, got 200
ok   unknown member        expected 404, got 404
ok   bad sort              expected 400, got 400
ok   write without key     expected 401, got 401
6/6 checks passed
```

The script behind it is a loop over a table of expectations:

JavaScript

```
async function check(label, path, expected, options = {}) {
  const response = await fetch(BASE + path, {
    headers: { Accept: "application/json",
               ...(options.key ? { "X-API-Key": KEY } : {}),
               ...options.headers },
    ...(options.body ? { method: options.method || "POST",
                         body: JSON.stringify(options.body) } : {}),
  });
  await response.text();
  const verdict = response.status === expected ? "ok  " : "FAIL";
  console.log(`${verdict} ${label.padEnd(22)} expected ${expected}, `
    + `got ${response.status}`);
  return response.status === expected;
}
```

Calling that function six times, with the expectations from the table in the concepts of this chapter, prints exactly the output at the top of this example.

Example 2: the real suite, top to bottom.

JavaScript

```
import test from "node:test";
import assert from "node:assert/strict";
import { createServer } from "./app.mjs";

const KEY = "lf-demo-key-123";
let base;

test.before(async () => {
  const server = createServer();
  await new Promise((resolve) => server.listen(0, "127.0.0.1", resolve));
  base = `http://127.0.0.1:${server.address().port}`;
});

test.after(() => server.close());

async function api(method, path, { body, headers = {} } = {}) {
  const options = { method, headers: { Accept: "application/json", ...headers } };
  if (body !== undefined) {
    options.headers["Content-Type"] = "application/json";
    options.body = JSON.stringify(body);
  }
  const response = await fetch(base + path, options);
  const text = await response.text();
  return { status: response.status, headers: response.headers,
           json: text ? JSON.parse(text) : null };
}

test("listing returns a page envelope plus a Link header", async () => {
  const res = await api("GET", "/api/v1/bookmarks?limit=5");
  assert.equal(res.status, 200);
  assert.equal(res.json.data.length, 5);
  assert.equal(res.json.meta.total, 12);
  assert.equal(res.json.meta.hasMore, true);
  assert.match(res.headers.get("link"), /rel="next"/);
});

test("a body that fails validation lists every bad field", async () => {
  const res = await api("POST", "/api/v1/bookmarks",
    { body: { url: "ftp://example.com" },
      headers: { "X-API-Key": KEY } });
  assert.equal(res.status, 400);
  assert.equal(res.json.status, 400);
  assert.deepEqual(res.json.errors.map((e) => e.field), ["title", "url"]);
});
```

output

```
(the file continues with ten more tests; run it with node --test)
```

Example 3: running the suite.

shell

```
node --test
```

output

```
TAP version 13
ok 1 - listing returns a page envelope plus a Link header
ok 2 - tag filter only keeps matching bookmarks
ok 3 - sort=title&order=desc really reverses the order
ok 4 - cursor pagination walks the collection without repeats
ok 5 - an unknown sort field is rejected with a problem document
ok 6 - writes need a valid API key
ok 7 - a body that fails validation lists every bad field
ok 8 - create, read, update and delete round trip
ok 9 - a duplicate URL is a conflict
ok 10 - the CORS preflight answers with the allowed methods
ok 11 - a token can be exchanged for a bearer session
ok 12 - the rate limiter answers 429 after the quota is spent
1..12
```

output

```
(the runner prints a YAML block of duration_ms, type and location under every
line, a "# Subtest: ..." line before each one, and a final block of
"# tests 12 / # pass 12 / # fail 0" counters; those lines are omitted here)
```

Twelve assertions, twelve passes, no dependencies installed.

Example 4: what a failure looks like, and why the wording matters.

Save the file below as `wrong-expectation.mjs` and run it on its own, so it does not join the suite in example 3.

shell

```
node --test wrong-expectation.mjs
```

JavaScript

```
import test from "node:test";
import assert from "node:assert/strict";
import { createServer } from "./app.mjs";

let server;
let base;

test.before(async () => {
  server = createServer();
  await new Promise((resolve) => server.listen(0, "127.0.0.1", resolve));
  base = `http://127.0.0.1:${server.address().port}`;
});

test.after(() => server.close());

test("creating a bookmark without a title succeeds", async () => {
  const response = await fetch(`${base}/api/v1/bookmarks`, {
    method: "POST",
    headers: { "Content-Type": "application/json",
               "X-API-Key": "lf-demo-key-123",
               Accept: "application/json" },
    body: JSON.stringify({ url: "https://example.com/no-title" }),
  });
  assert.equal(response.status, 201);
});
```

output

```
TAP version 13
not ok 1 - creating a bookmark without a title succeeds
  ---
  failureType: 'testCodeFailure'
  error: |-
    Expected values to be strictly equal:

    400 !== 201

  expected: 201
  actual: 400
  operator: 'strictEqual'
  name: 'AssertionError'
1..1
```

output

```
(the full output also carries a "# Subtest: ..." line before the failing one,
the duration_ms, type, location and stack frames, and a final block listing
tests, passes and failures; those lines are omitted here)
```

`400 !== 201` is the whole story: the test expected success, the server refused the body. Fixing the test is the wrong move here; the API is behaving correctly.

Example 5: a test that checks the problem document, not just the code.

JavaScript

```
test("an unknown sort field is rejected with a problem document", async () => {
  const res = await api("GET", "/api/v1/bookmarks?sort=colour");
  assert.equal(res.status, 400);
  assert.equal(res.json.type, "https://learnfree.dev/problems/invalid-query");
  assert.equal(res.json.errors[0].field, "sort");
  assert.match(res.headers.get("content-type"), /application\/problem\+json/);
});
```

output

```
ok 5 - an unknown sort field is rejected with a problem document
```

Asserting on `type` and on the content type catches a regression that a status code alone would miss: a server that answers `400` with a bare string body.

12.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Testing only the happy path	every error path is untested	one test per status code you document
Sharing one server between tests	order dependent failures	a fresh server per test file, on port 0
Asserting on a whole response body	a new field breaks every test	assert on the fields that matter
Leaving `console.log` in tests	noise hides the real failure	print only what failed

Mistake	Why it hurts	Best practice
Not testing rate limits	the first deploy discovers them for you	spawn a process with a tiny quota
Tests that talk to the internet	flaky, slow, sometimes forbidden	local fixtures, stubbed responses
Skipping the `OPTIONS` preflight test	browser clients fail in production	assert on `Allow-Methods`
No CI run	the suite rots within a week	run it on every push

- Name tests after the behaviour you expect, not the method you call.
- When a bug is reported, write the failing test first, then fix the code.

12.5 Practice exercises

1. Why should an API test suite start its own server instead of using a running one?
2. What are the three things a contract test should check, in order?
3. Why is `assert.deepEqual(page.errors.map(e => e.field), ["title","url"])` better than checking the status code alone?
4. How do you test a rate limit without waiting a minute?
5. A test asserts on an entire response body and fails after you add one field. What is wrong?

Solutions

1. So the suite controls the port, the data and the lifetime: no dependency on a machine state, and no port collisions between parallel files.
2. The status code, then the headers that carry policy such as `Location` or `Content-Type`, then the fields of the body.
3. Because it proves the server reported every invalid field, which is the behaviour clients rely on to fill in a form in one round trip.
4. Start the server in a child process with a very small quota, for example `RATE_LIMIT=3`, and send five requests.
5. The assertion is too wide: it encodes the entire shape, so any additive change looks like a failure. Assert on the fields the test is about.

12.6 Mini test

1. Which module ships Node's test runner in the standard library? A) `node:assert` B) `node:test` C) `node:http` D) `node:events`
2. Which port number asks the operating system for a free port? A) `80` B) `0` C) `3000` D) `9999`
3. What is the first thing a contract test should check? A) the response body B) the status code C) the response time D) the database row
4. True or false: a test suite should exercise error paths as well as the happy path.

5. Which assertion proves that a validation error named the right field?
6. Why is `assert.match(headers.get("content-type"), /problem\+json/)` a useful check?
7. What makes a `429` test fast rather than slow?

Answer key and explanations

1. B - `node:test` provides `test`, `before` and `after`.
2. B - port `0` means "any free port".
3. B - the status code decides whether the rest of the assertions mean anything.
4. True - clients break on undocumented error shapes more often than on happy path changes.
5. `assert.equal(res.json.errors[0].field, "title")`.
6. Because it checks the error is served as a problem document, which a status code alone cannot prove.
7. Running the server with a tiny quota in its own process, so a handful of requests is enough.

Chapter 13 - Consuming third-party APIs

13.1 Summary

- Read the documentation first, then write a request, then automate it.
- Keep credentials in environment variables, never in the source file.
- Set a timeout on every call; a hung request is worse than a failed one.
- Retry only what is safe to retry: network errors, `429` and `5xx`.
- Use exponential backoff with jitter so many clients do not retry in lockstep.
- Honour `Retry-After` when the server sends it.
- Follow the API's own pagination fields instead of guessing.
- Treat any field you read as optional until the documentation says otherwise.
- Log the status and the latency of every call; that is how you spot a degraded API.
- Wrap the API in a small module of your own so the rest of the application never sees HTTP details.

13.2 Concepts

The wrapper pattern

output

```
your application                your module              the remote API
----------------                -----------               --------------
listAllBookmarks()      ->      fetch with timeout  ->   GET /bookmarks
                                retry with backoff
                                follow the cursor
                                normalise the shape       <- one page
                                return plain objects
```

The rest of the program calls `listAllBookmarks()` and never learns whether the remote service uses cursors, links or offsets. When the remote API changes, one file changes.

Timeouts, retries and backoff

Situation	Retry?	Wait
Connection refused	yes	exponential backoff
Timeout	careful: the request may have arrived	backoff, and make the call idempotent
`429`	yes	`Retry-After`
`500`, `502`, `503`	yes	exponential backoff
`400`, `401`, `403`, `404`, `409`	no	fix the request

output

```
wait = min(base * 2^(attempt-1), cap)     then add jitter
attempt 1 -> 200 ms      attempt 3 -> 800 ms      attempt 5 -> 3200 ms
attempt 2 -> 400 ms      attempt 4 -> 1600 ms     attempt 6 -> cap (5000 ms)
```

Without jitter, a thousand clients that failed together retry together and fail again. Adding a random fraction of the wait spreads the load.

13.3 Examples

Example 1: a careful client for the demo API.

JavaScript

```
const BASE = process.env.BASE || "http://127.0.0.1:3000/api/v1";
const KEY = process.env.API_KEY || "lf-demo-key-123";
const sleep = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

async function fetchJson(url, options = {}) {
  const { timeout = 3000, retries = 3, headers = {} } = options;
  let lastError = null;
  for (let attempt = 1; attempt <= retries; attempt += 1) {
    const started = Date.now();
    try {
      const response = await fetch(url, {
        headers: { Accept: "application/json", "X-API-Key": KEY, ...headers },
        signal: AbortSignal.timeout(timeout),
      });
      const text = await response.text();
      const shown = new URL(url);
      console.log(`attempt ${attempt}: ${response.status} `
        + `${shown.pathname}${shown.search} (${Date.now() - started} ms)`);
      if (response.status === 429 || response.status >= 500) {
        const retryAfter = Number(response.headers.get("retry-after") || 1);
        const wait = response.status === 429
          ? retryAfter * 1000
          : Math.min(200 * 2 ** (attempt - 1), 5000);
        console.log(`  retryable, waiting ${wait} ms`);
        lastError = new Error(`status ${response.status}`);
        await sleep(wait);
        continue;
      }
      return { status: response.status, headers: response.headers,
               body: text ? JSON.parse(text) : null };
    } catch (error) {
      console.log(`attempt ${attempt}: network error ${error.name} `
        + `(${Date.now() - started} ms)`);
      lastError = error;
      await sleep(Math.min(200 * 2 ** (attempt - 1), 5000));
    }
  }
  throw lastError;
}
```

output

```
(the function is used by the two runs below; it returns the parsed body or
throws the last error)
```

Example 2: walking the collection.

shell

```
node client.mjs
```

output

```
attempt 1: 200 /api/v1/bookmarks?limit=4 (235 ms)
page 1: 4 bookmarks, total 12
attempt 1: 200 /api/v1/bookmarks?limit=4&cursor=eyJvZmZzZXQiOjR9 (17 ms)
page 2: 4 bookmarks
attempt 1: 200 /api/v1/bookmarks?limit=4&cursor=eyJvZmZzZXQiOjh9 (7 ms)
page 3: 4 bookmarks
attempt 1: 200 /api/v1/bookmarks/bm_001 (8 ms)
single read: MDN HTTP reference
walked 3 pages with retries in place
```

Every attempt prints the status and the latency, so a slow page is visible without adding instrumentation. The timings differ from machine to machine.

Example 3: retrying a `429` correctly.

shell

```
node client.mjs
```

output

```
attempt 1: 200 /api/v1/bookmarks?limit=4 (457 ms)
page 1: 4 bookmarks, total 12
attempt 1: 429 /api/v1/bookmarks?limit=4&cursor=eyJvZmZzZXQiOjR9 (20 ms)
  retryable, waiting 2000 ms
attempt 2: 200 /api/v1/bookmarks?limit=4&cursor=eyJvZmZzZXQiOjR9 (8 ms)
page 2: 4 bookmarks
attempt 1: 429 /api/v1/bookmarks?limit=4&cursor=eyJvZmZzZXQiOjh9 (4 ms)
  retryable, waiting 2000 ms
attempt 2: 200 /api/v1/bookmarks?limit=4&cursor=eyJvZmZzZXQiOjh9 (6 ms)
page 3: 4 bookmarks
attempt 1: 429 /api/v1/bookmarks/bm_001 (7 ms)
  retryable, waiting 2000 ms
attempt 2: 200 /api/v1/bookmarks/bm_001 (7 ms)
single read: MDN HTTP reference
walked 3 pages with retries in place
```

This run points at a copy of the API started with a quota of one request per two seconds, so every second request is refused. The client reads `Retry-After: 2`, waits 2000 ms and succeeds. Against the normal server, the same script prints only `200` lines.

Example 4: retrying a connection that was refused.

shell

```
BASE=http://127.0.0.1:3999/api/v1 node client.mjs
```

output

```
attempt 1: network error TypeError (155 ms)
attempt 2: network error TypeError (6 ms)
attempt 3: network error TypeError (5 ms)
```

output

```
(the script then throws the last error and exits with a non zero code; the
gap between attempts is the exponential backoff: about 200 ms, then 400 ms)
```

Nothing listens on port 3999. Three attempts, three logs, one exception, no infinite loop.

Example 5: the schedule those waits come from.

JavaScript

```
const baseMs = 200;
const capMs = 5000;
const pad = (value, width) => String(value).padEnd(width);
console.log("attempt  exponential  with jitter");
for (let attempt = 1; attempt <= 6; attempt += 1) {
  const exponential = Math.min(baseMs * 2 ** (attempt - 1), capMs);
  const jitter = Math.round(Math.random() * exponential);
  console.log(`${pad(attempt, 8)}${pad(exponential + " ms", 13)}${jitter} ms`);
}
```

output

```
attempt  exponential  with jitter
1       200 ms       112 ms
2       400 ms       159 ms
3       800 ms       116 ms
4       1600 ms      275 ms
5       3200 ms      568 ms
6       5000 ms      3554 ms
```

The exponential column is deterministic; the jitter column is random, which is the point.

13.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
No timeout	one hung call freezes your feature	`AbortSignal.timeout(3000)` on every request
Retrying a `400`	the same request fails forever	retry only `429` and `5xx` and network errors
Retrying without backoff	you amplify the outage	exponential backoff with jitter
Ignoring `Retry-After`	the service bans you	honour it, and cap your own rate
Keys in the source file	they leak with the repository	environment variables, loaded at startup
Hard-coding the cursor's meaning	your client breaks on a server change	treat it as an opaque string
Reading a field the docs mark optional	a crash on someone else's deploy	defensive access, logged
Retrying a non-idempotent POST blindly	duplicates downstream	send an idempotency key

- Log the status and the latency of every outbound call; you will need it the first time the provider degrades.
- Write a thin module per provider, and unit test it against recorded responses.

13.5 Practice exercises

1. Which responses are safe to retry, and which are not?

2. Why must a client honour `Retry-After` rather than using its own backoff on a `429`?
3. What does `AbortSignal.timeout(3000)` do?
4. Why add jitter to a backoff schedule?
5. Where should an API key live in a Node project?

Solutions

1. Safe: network errors, `429` and `5xx`. Not safe: `400`, `401`, `403`, `404`, `409`, which will fail identically on every attempt.
2. Because `Retry-After` is the server telling you exactly when its quota window resets; a shorter wait just gets you refused again.
3. It aborts the request if no response has arrived within three seconds, so a hung socket cannot block the caller indefinitely.
4. So that clients which failed at the same moment do not all retry at the same moment, which would keep the service down.
5. In an environment variable, read once at startup, never committed and never printed in logs.

13.6 Mini test

1. Which status code tells a client to slow down? A) `400` B) `401` C) `429` D) `500`
2. Which API aborts a `fetch` after three seconds? A) `fetch(url, { signal: AbortSignal.timeout(3000) })` B) `fetch(url, { timeout: 3000 })` C) `setTimeout(fetch, 3000)` D) `fetch(url, { abortAfter: 3000 })`
3. What is exponential backoff? A) retrying immediately B) doubling the wait between attempts, up to a cap C) retrying once only D) waiting a fixed second
4. True or false: a `400` response should be retried after a pause.
5. Why is a cursor an opaque string for a client?
6. What does `Retry-After: 60` mean?
7. Why should an outbound call be logged with its status and latency?

Answer key and explanations

1. C - `429 Too Many Requests` is the quota answer.
2. A - `AbortSignal.timeout` produces a signal that aborts the request.
3. B - each attempt waits twice as long, capped so the client does not wait forever.
4. False - the request itself is wrong; retrying repeats the same failure.
5. Because its contents are the server's business; parsing it couples the client to internals that may change without notice.
6. Wait 60 seconds before sending the next request.
7. Because the first symptom of a failing dependency is a slow or failing response, and the log is the only place that is visible.

Chapter 14 - Capstone: design, document and test a bookmarks API

14.1 Summary

- The capstone puts the whole volume into one small service: a public bookmark store.
- The resource model has one collection, one member, one token endpoint and one identity endpoint.
- Every decision in it comes from a chapter of this volume, which is the point of the exercise.
- The server is three files of dependency-free Node, so nothing can go out of date on your machine.
- The description is one OpenAPI 3.1 document that parses, lints clean and matches the routes.
- The suite is twelve `node:test` cases covering status codes, headers, problem documents and the quota.
- A client module demonstrates timeouts, retries and backoff against the same service.
- Transport headers (`X-Request-Id`), security headers and CORS are present from the first request.
- The whole thing runs offline on `127.0.0.1:3000`, which is what makes it a reliable teaching artefact.
- When you build your own API, copy the shape of this one before you invent anything new.

14.2 Concepts

The brief

A user keeps bookmarks. Other programs must be able to list them, filter them, page through them, add one, edit one and delete one, all with a credential, all documented, all tested.

The resource model

output

```
/api/v1
  health                    GET     liveness, no credential
  bookmarks                 GET     list, filter, sort, paginate
  bookmarks                 POST    create, needs X-API-Key
  bookmarks/{bookmarkId}    GET     read one
  bookmarks/{bookmarkId}    PATCH   partial update, needs X-API-Key
  bookmarks/{bookmarkId}    DELETE  remove, needs X-API-Key
  tokens                    POST    exchange an API key for a bearer token
  me                        GET     who the token belongs to
  OPTIONS /bookmarks                CORS preflight
```

The decisions, and where they came from

Decision	Value	Chapter
Transport	HTTP/1.1, JSON bodies	1
Style	resources, no verbs in the path	2, 3
Plural collections	`/bookmarks`	3
Nesting	none: filters are query parameters	3
Methods	`GET`, `POST`, `PATCH`, `DELETE`	4
Unsupported method	`405` with `Allow`	4
Success codes	`200`, `201`, `204`	5
Errors	RFC 9457 problem documents	5
Field style	`camelCase`, ISO 8601 timestamps	6
Filters	`tag`, `favorite`, `q`, `sort`, `order`	7
Pagination	offset or cursor, plus a `Link` header	8
Auth	API key, then a JWT bearer token	9
Description	OpenAPI 3.1 with shared schemas	10
Versioning	`/api/v1` in the path, `/api/v2` beside it	11
Quota	100 requests per minute, headers included	11
CORS	an explicit origin, a preflight, `Max-Age`	11
Tests	`node:test`, twelve cases	12
Client	timeout, retry, backoff, `Retry-After`	13

14.3 Examples

Example 1: the files you need.

output

```
api-check/
  app.mjs                the routes, the store and the helpers
  server.mjs             listens on port 3000
  bodies/                request payloads used by the curl examples
  openapi.yaml           the OpenAPI 3.1 description
  lint_openapi.py        the lint script used in chapter 10
  bookmarks.test.mjs     the automated suite
  client.mjs             the careful consumer from chapter 13
  smoke.mjs              the five second health check
  jwt-parts.mjs          prints the three parts of a JWT
  flood.mjs              sends requests until the quota answers 429
  capstone.mjs           the run shown in example 2
```

shell

```
node server.mjs
node --test
node client.mjs
python lint_openapi.py openapi.yaml
```

output

```
bookmarks API listening on http://127.0.0.1:3000
```

Nothing is installed: the server uses only `node:http` and `node:crypto`, and the tests use only `node:test`.

Example 2: the whole life of a resource, in one run.

shell

```
node capstone.mjs
```

output

```
1. create
   status  : 201
   location: http://127.0.0.1:3000/api/v1/bookmarks/bm_013
   id      : bm_013
2. read back
   status  : 200 -> OpenAPI examples
3. patch
   status  : 200 -> favorite: true title kept: OpenAPI examples
4. list with the new row
   status  : 200 -> first: OpenAPI examples total: 13
5. delete
   status  : 204
6. read after delete
   status  : 404 -> Bookmark not found
7. list again
   status  : 200 -> total: 12
```

`201` with a `Location`, a `PATCH` that keeps the fields it was not sent, a `204` with no body, a `404` problem document afterwards, and a total that goes up and back down. The store is in memory, so a restart resets it to twelve bookmarks.

Example 3: the description, linted.

shell

```
python lint_openapi.py openapi.yaml
```

output

```
checked openapi.yaml
paths      : 5
operations : 8
schemas    : 5
references : 28
PROBLEMS   : 0
```

Five paths, eight operations, five schemas, twenty-eight internal references, no problems. The same document tells a code generator everything it needs to produce a client for these routes.

Example 4: the health check and the suite.

shell

```
node smoke.mjs
```

output

```
ok   health                expected 200, got 200
ok   collection            expected 200, got 200
ok   member                expected 200, got 200
ok   unknown member        expected 404, got 404
ok   bad sort              expected 400, got 400
ok   write without key     expected 401, got 401
6/6 checks passed
```

shell

```
node --test
```

output

```
TAP version 13
ok 1 - listing returns a page envelope plus a Link header
ok 2 - tag filter only keeps matching bookmarks
ok 3 - sort=title&order=desc really reverses the order
ok 4 - cursor pagination walks the collection without repeats
ok 5 - an unknown sort field is rejected with a problem document
ok 6 - writes need a valid API key
ok 7 - a body that fails validation lists every bad field
ok 8 - create, read, update and delete round trip
ok 9 - a duplicate URL is a conflict
ok 10 - the CORS preflight answers with the allowed methods
ok 11 - a token can be exchanged for a bearer session
ok 12 - the rate limiter answers 429 after the quota is spent
1..12
```

Example 5: the checklist before you publish.

output

```
[ ] every route returns a documented status code
[ ] every error is a problem document with a type URI
[ ] the OpenAPI file parses and lints clean
[ ] writes require a credential and say so with WWW-Authenticate
[ ] rate limit headers are on every response
[ ] the preflight answers on routes a browser will call
[ ] tokens carry an expiry and are never cached
[ ] CORS names an explicit origin, not a wildcard
[ ] the suite runs in CI and passes
[ ] a client outside your network can read the docs page
```

The list is deliberately boring. Everything on it is a decision you make once and be done with, and each one is a paragraph you will not have to write in an incident report later.

14.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Documenting after shipping	the document describes a service nobody built	description and routes in the same commit
Error shapes that vary per route	every client needs special cases	one problem document, one `type` namespace
No `X-Request-Id`	you cannot trace a report	generate one per request, echo it
Credentials accepted from anywhere	tokens leak through URLs and logs	headers only, and never logged
Pagination without a total	clients cannot show progress	`total`, or `hasMore` if a count is too expensive
Tests that need the internet	the suite becomes flaky	local server, ephemeral port
A wildcard CORS origin	any site can call your API	an explicit allow list

- Re-read your own route table once a week for the first month; that is when the naming mistakes show.
- When a client complains, reproduce it with `curl.exe -s -i` before touching code.

14.5 Practice exercises

1. Add a `GET /api/v1/bookmarks/{bookmarkId}/tags` route. What is wrong with that design, and what would you do instead?
2. Which four files make up the minimal version of this service?
3. Which response header tells a client which credential scheme to use?
4. The suite must not share state with a server you started by hand. How does the test file achieve that?
5. Why does the API answer `405` with an `Allow` header instead of `404` for an unsupported method?

Solutions

1. It adds a third level of nesting for data that is already a field of the bookmark. Keep `tags` inside the representation and offer `?tag=http` on the collection for filtering.
2. `app.mjs`, `server.mjs`, `openapi.yaml` and `bookmarks.test.mjs`; the rest of the files are demonstrations of the surrounding chapters.
3. `WWW-Authenticate`, for example `ApiKey realm=bookmarks` or `Bearer realm="bookmarks"`.
4. It imports `createServer()` and listens on port `0`, so the operating system assigns a free port, and closes the server in `test.after`.
5. Because the path exists: `404` would tell the client there is no such resource at all, while `405` says "this resource is here, use another verb", and `Allow` names the verbs.

14.6 Mini test

1. Which status code does the API send when a `POST` creates a bookmark? A) `200` B) `201` C) `204` D) `202`
2. Which header appears on a `201` response and points at the new resource? A) `Location` B) `Link` C) `Retry-After` D) `Allow`
3. How many operations does the demo OpenAPI document declare? A) five B) eight C) twelve D) twenty eight
4. True or false: the demo API stores its bookmarks in a database.
5. Which header carries the trace identifier of a request?
6. Why does the test file listen on port `0`?
7. What does the client do when it receives a `429` with `Retry-After: 2`?

Answer key and explanations

1. B - `201 Created` marks a new resource; the `Location` header names it.
2. A - `Location` is the standard header for a newly created resource.
3. B - eight operations across five paths, as the lint output shows.
4. False - the store is an array in memory, which is why a restart resets the data.
5. `X-Request-Id`, a fresh UUID per request.
6. So the operating system picks a free port and the suite never collides with a server you are running by hand.
7. It waits two seconds, as the header instructs, and then sends the same request again.

Cheat sheet

15.1 Requests and responses

Part	Request	Response
Line	`GET /api/v1/bookmarks HTTP/1.1`	`HTTP/1.1 200 OK`
Metadata	headers	headers
Payload	body, usually JSON	body, usually JSON
Format	`Content-Type`	`Content-Type`

15.2 Methods

Method	Safe	Idempotent	Body	Success
`GET`	yes	yes	no	`200`
`HEAD`	yes	yes	no	`200`

Method	Safe	Idempotent	Body	Success
`OPTIONS`	yes	yes	optional	`204`
`POST`	no	no	yes	`201`
`PUT`	no	yes	yes	`200`, `201`
`PATCH`	no	yes	yes	`200`
`DELETE`	no	yes	optional	`204`

15.3 Status codes

Code	Use when	Code	Use when
`200`	read or update worked	`409`	collides with existing state
`201`	created; send `Location`	`412`	an `If-Match` guard failed
`202`	queued for later	`415`	the media type is wrong
`204`	done, nothing to return	`422`	valid JSON, invalid meaning
`304`	nothing new since	`429`	quota exhausted
`400`	malformed or invalid input	`500`	your bug
`401`	credential missing or invalid	`503`	temporarily unavailable
`403`	known caller, not allowed		
`404`	nothing matches the path		
`405`	path exists, verb does not		

15.4 Problem documents

JSON

```json
{
  "type": "https://example.com/problems/validation-error",
  "title": "Request body failed validation",
  "status": 400,
  "detail": "1 field is invalid",
  "instance": "/api/v1/bookmarks",
  "errors": [
    { "field": "title", "code": "required", "message": "a title is required" }
  ]
}
```

15.5 Query parameters

Purpose	Parameters
Filter	`tag`, `favorite`, `q`, `createdAfter`
Sort	`sort=createdAt`, `order=asc`
Page	`limit`, `offset`, `cursor`
Shape	`fields`

15.6 Pagination envelopes

JSON

```json
{
  "data": [],
  "meta": { "limit": 20, "offset": 0, "total": 12,
            "hasMore": true, "nextCursor": "eyJvZmZzZXQiOjIwfQ" }
}
```

15.7 Security and quota headers

Header	Meaning
`Authorization: Bearer <jwt>`	the caller's token
`X-API-Key: <key>`	a project key
`WWW-Authenticate`	the scheme the server expects
`X-RateLimit-Limit`	requests allowed per window
`X-RateLimit-Remaining`	requests left in this window
`X-RateLimit-Reset`	unix time the window resets
`Retry-After`	seconds to wait before retrying
`Cache-Control: no-store`	never keep a copy
`X-Request-Id`	the id of this request

15.8 CORS and security headers

Header	Meaning
`Access-Control-Allow-Origin`	the one origin allowed to read the answer
`Access-Control-Allow-Methods`	verbs the preflight may allow
`Access-Control-Allow-Headers`	headers the browser may send
`Access-Control-Max-Age`	seconds the permission may be cached
`Vary: Origin`	the answer depends on the caller

Header	Meaning
`X-Content-Type-Options: nosniff`	do not guess the body type
`Referrer-Policy: no-referrer`	do not leak the URL

15.9 OpenAPI skeleton

YAML

```yaml
openapi: 3.1.0
info: { title: Bookmarks API, version: 1.0.0 }
servers: [{ url: http://127.0.0.1:3000/api/v1 }]
security: [{ apiKey: [] }]
paths:
  /bookmarks:
    get:
      operationId: listBookmarks
      parameters:
        - { name: limit, in: query,
            schema: { type: integer, minimum: 1, maximum: 100, default: 20 } }
      responses:
        "200": { description: A page of bookmarks }
components:
  securitySchemes:
    apiKey: { type: apiKey, in: header, name: X-API-Key }
  schemas: {}
```

15.10 Client recipe

JavaScript

```javascript
const response = await fetch(url, {
  headers: { Accept: "application/json", "X-API-Key": KEY },
  signal: AbortSignal.timeout(3000),
});
if (response.status === 429) {
  await sleep(Number(response.headers.get("retry-after") || 1) * 1000);
}
```

Final test

1. Which side of an HTTP exchange starts the conversation?
2. Name the four parts of an HTTP request.
3. Which REST constraint requires that every request carry everything the server needs?
4. Which path is the best name for a single bookmark? A) `/api/v1/getBookmark/bm_003` B) `/api/v1/bookmark?id=bm_003` C) `/api/v1/bookmarks/bm_003` D) `/api/v1/bookmarks/bm_003.json`
5. True or false: nesting depth in a URL should mirror the database joins.
6. Which methods are safe, and why does that matter?
7. What does idempotent mean, and which of `GET`, `PUT`, `POST`, `DELETE` is not idempotent?

8. A `PATCH` body contains only `{"favorite":true}`. What must happen to the other fields?
9. Which status code answers a known path with an unsupported method, and which header accompanies it?
10. Name the four fields of an RFC 9457 problem document.
11. Which status code means the caller is known but not allowed, and which one means the credential is missing?
12. Which content type should an API use for problem documents?
13. What is the difference between `null` and an absent key in a JSON body?
14. Which timestamp format should a JSON API use?
15. Which query parameters would you use to list the two most recent bookmarks tagged `http`?
16. Which two problems does offset pagination suffer from on a large, changing collection?
17. What does a cursor represent to the client, and what must the client never do with it?
18. Name the three parts of a JWT and say which one is signed.
19. Which OAuth 2.0 flow should a single page application use, and what does PKCE add?
20. Why must a credential never be placed in a URL?
21. Which top level key of an OpenAPI document holds reusable schemas?
22. Which two values does the `enum` keyword constrain in a query parameter?
23. Which header on a `429` response tells the client how long to wait?
24. What does a CORS preflight answer with, and why does it matter?
25. Which status codes may a client safely retry, and which backoff schedule should it use?

16.1 Final test answers

1. The client. The client builds and sends the request; the server reads it, acts and answers.
2. Method, target (path plus query), headers and body. The response has the status line, headers and a body.
3. Statelessness: the server keeps no memory of previous requests, so each request must be self contained.
4. C - plural collection, member identifier, no verb and no file extension.
5. False - the URL describes the API's resource model, not the schema; joins are an implementation detail and should be flattened with a query parameter.
6. `GET`, `HEAD` and `OPTIONS`. Because they change nothing, caches, prefetchers and crawlers may repeat them at any time without harm.
7. Repeating the request leaves the server in the same state; `POST` is the one that is not idempotent, because each call can create another resource.
8. They keep their current values. A patch merges, and a body that resets omitted fields is a `PUT` wearing the wrong method.
9. `405 Method Not Allowed`, accompanied by an `Allow` header listing the methods the resource does accept.

10. `type`, `title`, `status` and `detail`.
11. `403 Forbidden` means the caller is known but not allowed; `401` means the credential is missing or invalid.
12. `application/problem+json`.
13. `null` means the field exists and its value is nothing; an absent key means the field was not part of the document, which is what lets a `PATCH` leave it untouched.
14. ISO 8601 with an explicit offset, for example `2026-09-03T08:40:00.000Z`.
15. `?tag=http&sort=createdAt&order=desc&limit=2`.
16. Pages drift when rows are inserted or deleted, and deep offsets force the database to read and discard everything before the page.
17. A position in a deterministic order. The client must treat it as an opaque token and never decode or construct one, because its contents are the server's business.
18. Header, payload and signature. The signature covers the first two parts and proves they were not modified; the payload is only encoded, not encrypted.
19. Authorization code with PKCE. PKCE binds the code exchange to the client that started it, by sending a challenge up front and the matching verifier only during the exchange.
20. Because URLs are recorded in access logs, kept in browser history and sent as referrers, so a credential in a URL is a credential you have published.
21. `components`.
22. The exact set of values a query parameter may take, such as `enum: [createdAt, title, url]` for `sort`.
23. `Retry-After`, expressed in seconds.
24. The allowed origin, the allowed methods, the allowed headers and how long the permission may be cached. Without it, a browser refuses to send a request that uses a JSON body or a custom header.
25. `429` and `5xx`, plus network errors and timeouts. The schedule is exponential backoff with jitter, capped, and `Retry-After` is honoured whenever the server sends it.

Sources

Consulted while writing this volume. The syllabus planned for this volume came from a course hub that now requires a login and answers with a 404, so the free official documentation below was used instead.

- MDN Web Docs, HTTP overview: https://developer.mozilla.org/en-US/docs/Web/HTTP
- MDN, HTTP methods: https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods
- MDN, HTTP response status codes: https://developer.mozilla.org/en-US/docs/Web/HTTP/Status
- MDN, HTTP headers: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers
- MDN, the `Link` header: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Link

- MDN, the `Retry-After` header: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Retry-After
- MDN, Cross-Origin Resource Sharing (CORS): https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS
- MDN, the Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- MDN, `AbortSignal.timeout()`: https://developer.mozilla.org/en-US/docs/Web/API/AbortSignal/timeout_static
- RFC 9110, HTTP Semantics: https://www.rfc-editor.org/rfc/rfc9110.html
- RFC 9457, Problem Details for HTTP APIs: https://www.rfc-editor.org/rfc/rfc9457.html
- OpenAPI Specification, version 3.1: https://spec.openapis.org/oas/latest.html
- REST API tutorial, resource naming and status codes: https://restfulapi.net/
- JSON:API, conventions for JSON payloads and errors: https://jsonapi.org/
- Node.js, the HTTP module: https://nodejs.org/api/http.html
- Node.js, the test runner: https://nodejs.org/api/test.html
- Node.js, the crypto module: https://nodejs.org/api/crypto.html

The demo bookmarks API, its OpenAPI description, its test suite and every transcript in this volume are original work written for this volume and run locally against `http://127.0.0.1:3000`.